

ACTIVIDAD LABORATORIO NO.2
TRABAJO: DESAMBIGUACIÓN DEL SENTIDO DE LAS PALABRAS USANDO NLTK Y
APRENDIZAJE AUTOMÁTICO SUPERVISADO

PRESENTADO POR:
VIVIANA ANDREA BAUTISTA PULIDO
CARMEN EDILIA RICARDO GELVES
ALEJANDRO DE MENDOZA

PRESENTADO AL PROFESOR:
ING DIEGO OSORIO REINA

FUNDACIÓN UNIVERSITARIA INTERNACIONAL DE LA RIOJA
BOGOTÁ D.C.
28 DE JUNIO
2025

TABLA DE CONTENIDO

INTRODUCCIÓN.....	3
TÍTULO DEL PROYECTO	3
DESARROLLO DEL PROYECTO	3
Descripción de la actividad	4
Fases del desarrollo:.....	4
1. Análisis del Corpus:	4
2. Extracción de Características	5
3. Entrenamiento y Evaluación.....	6
4. Reflexión Crítica	8
5. Explicación De Todo El Código	9
CONCLUSIONES DE LABORATORIO.....	49
BIBLIOGRAFÍA.....	51

INTRODUCCIÓN

El propósito de este laboratorio es aplicar técnicas de aprendizaje automático supervisado para abordar el problema de la desambiguación del sentido de palabras (Word Sense Disambiguation, WSD), utilizando el corpus etiquetado Senseval 2, disponible en el paquete NLTK. Las palabras seleccionadas para el análisis son "hard" y "serve", debido a su alta polisemia y su relevancia en contextos lingüísticos diversos. El objetivo principal es desarrollar y evaluar clasificadores basados en el algoritmo Naive Bayes, explorando diferentes conjuntos de características y analizando su impacto en el rendimiento del modelo.

El proceso incluye las siguientes etapas clave:

- Análisis exploratorio del corpus: Estudio de los sentidos y las formas gramaticales de las palabras objetivo, identificando su distribución y contextos de uso en el corpus Senseval 2.
- Preprocesamiento y limpieza de datos: Identificación y corrección de posibles inconsistencias en el corpus, como datos faltantes, ruido o anotaciones ambiguas.
- Extracción de características: Implementación de enfoques como bolsa de palabras (Bag of Words) y colocaciones, así como la exploración de características contextuales y morfosintácticas relevantes.
- Entrenamiento y validación de clasificadores: Construcción de modelos Naive Bayes, ajustados con diferentes combinaciones de características, y evaluación de su desempeño utilizando métricas cuantitativas como precisión, sensibilidad, especificidad y F1-score.
- Análisis de errores y reflexiones: Identificación de patrones en los errores de clasificación, discusión sobre las limitaciones de los enfoques utilizados y propuestas de mejora para futuros experimentos.

La implementación se llevó a cabo en Python 3.9.13 utilizando el entorno interactivo de Jupyter Notebook, aprovechando las funcionalidades de la biblioteca NLTK 3.3 para el procesamiento del lenguaje natural y el manejo del corpus.

Este laboratorio busca no solo evaluar el rendimiento de los clasificadores, sino también profundizar en los desafíos inherentes a la desambiguación del sentido de palabras y su relevancia en aplicaciones de procesamiento del lenguaje natural.

TÍTULO DEL PROYECTO

Para el desarrollo de esta actividad elegimos como título del proyecto **“DESAMBIGUACIÓN DEL SENTIDO DE LAS PALABRAS USANDO NLTK Y APRENDIZAJE AUTOMÁTICO SUPERVISADO”**.

DESARROLLO DEL PROYECTO

En esta sección se detalla el proceso técnico llevado a cabo para abordar la tarea de desambiguación del sentido de las palabras (WSD) utilizando el corpus Senseval 2 del paquete NLTK y el algoritmo de aprendizaje automático Naive Bayes. El desarrollo del laboratorio se estructura en una serie de etapas metodológicas que abarcan desde el análisis inicial del corpus hasta la evaluación de los clasificadores y el análisis de resultados. Las palabras seleccionadas para el estudio, "hard" y "serve", se analizan en sus diferentes sentidos y contextos, con el objetivo de implementar y comparar clasificadores basados en distintos conjuntos de características lingüísticas.

El proceso se implementó en Python 3.9.13 utilizando Jupyter Notebook como entorno de desarrollo, aprovechando las funcionalidades de la biblioteca NLTK 3.3 para el procesamiento del lenguaje natural. Las principales actividades realizadas incluyen:

1. **Carga y exploración del corpus:** Se accede al corpus Senseval 2 para extraer los contextos y sentidos etiquetados de las palabras objetivo, analizando su distribución y características gramaticales.
2. **Preprocesamiento de datos:** Se aplican técnicas de limpieza, como tokenización, eliminación de palabras vacías y normalización de texto, para preparar los datos para la extracción de características.
3. **Definición y extracción de características:** Se implementan dos enfoques principales: bolsa de palabras (Bag of Words) y colocaciones, complementados con información morfosintáctica (etiquetas POS) para capturar el contexto de las palabras.
4. **Entrenamiento de clasificadores:** Se dividen los datos en conjuntos de entrenamiento y prueba, utilizando el algoritmo Naive Bayes para construir modelos que clasifiquen los sentidos de "hard" y "serve".
5. **Evaluación del rendimiento:** Se calculan métricas como precisión, sensibilidad y F1-score, empleando validación cruzada para garantizar resultados robustos.
6. **Análisis de resultados:** Se identifican los errores de clasificación y se exploran las limitaciones de los modelos, proponiendo posibles mejoras.

Este desarrollo busca no solo cumplir con los objetivos del laboratorio, sino también proporcionar una experiencia práctica en el manejo de herramientas de PLN y en la aplicación de técnicas de aprendizaje automático a problemas reales de procesamiento lingüístico. Cada etapa se documenta paso a paso, incluyendo fragmentos de código relevantes y visualizaciones para facilitar la comprensión de los resultados.

Descripción de la actividad

Se analizaron dos palabras polisémicas en inglés, "hard" y "serve", seleccionadas por su alta ambigüedad semántica y su frecuencia en diversos contextos lingüísticos dentro del corpus Senseval 2 de la biblioteca NLTK 3.3. El objetivo principal fue diseñar, entrenar y evaluar clasificadores basados en el algoritmo Naive Bayes para determinar el sentido correcto de estas palabras en diferentes oraciones, utilizando múltiples conjuntos de características lingüísticas.

Para ello, se implementaron dos enfoques principales de extracción de características:

- **Características basadas en contexto:** Se empleó el modelo de bolsa de palabras (Bag of Words), que considera las palabras circundantes en un ventana de contexto específica, y se complementó con etiquetas morfosintácticas (POS tags) para capturar información gramatical relevante.
- **Características basadas en colocaciones:** Se identificaron pares de palabras que co-ocurren frecuentemente con "hard" y "serve", analizando su relevancia semántica para mejorar la capacidad del clasificador de distinguir entre sentidos.

El proceso incluyó la división del corpus en conjuntos de entrenamiento y prueba, el entrenamiento de los modelos Naive Bayes utilizando la biblioteca NLTK en Python 3.9.13, y la evaluación de su rendimiento mediante métricas estándar como precisión, sensibilidad y F1-score. Este enfoque permitió comparar la efectividad de las diferentes configuraciones de características y analizar su impacto en la desambiguación del sentido de las palabras en contextos reales.

Fases del desarrollo:

1. **Análisis del Corpus:**

Para iniciar el análisis del corpus Senseval 2 utilizando la biblioteca NLTK 3.3 en Python 3.9.13, se llevaron a cabo las siguientes tareas fundamentales para caracterizar las palabras objetivo "hard" y "serve" y preparar los datos para la desambiguación del sentido de las palabras (WSD):

- **Identificación de sentidos:** Se extrajeron y catalogaron los sentidos asociados a cada palabra en el corpus, según las etiquetas predefinidas en Senseval 2:
 - **"hard":** Incluye los sentidos HARD1 (dureza física, ej. "superficie dura"), HARD2 (dificultad, ej. "tarea difícil") y HARD3 (esfuerzo o severidad, ej. "trabajar duro").
 - **"serve":** Abarca los sentidos SERVE10 (servir comida o bebida, ej. "servir la cena"), SERVE12 (atender o asistir, ej. "servir a un cliente"), SERVE2 (cumplir una función, ej. "servir como ejemplo") y SERVE6 (acción en deportes, ej. "servir en tenis"). Este paso permitió comprender la diversidad semántica de las palabras y su representación en el corpus.
- **Conteo de instancias por sentido:** Se cuantificó la distribución de instancias para cada sentido en el corpus, generando estadísticas que reflejan la frecuencia de cada uso. Este análisis reveló posibles desbalances en los datos (por ejemplo, si un sentido tiene significativamente más instancias que otro), lo cual podría influir en el rendimiento del clasificador Naive Bayes.
- **Identificación de formas gramaticales:** Se examinaron las diferentes formas gramaticales de las palabras objetivo que están presentes en el corpus. Para "hard", se identificaron las formas hard (base), harder (comparativo) y hardest (superlativo), junto con sus respectivos contextos. Para "serve", se consideraron formas como serve, served, serving y serves, analizando sus roles gramaticales (verbo, sustantivo en el caso de "serve" en deportes). Este análisis se apoyó en las etiquetas de partes del discurso (POS tags) proporcionadas por NLTK.
- **Detección de inconsistencias:** Se realizó una inspección exhaustiva para identificar instancias mal formateadas o con etiquetas incorrectas en el corpus. Por ejemplo, se detectaron casos donde las anotaciones de sentido no coincidían con el contexto de la oración o donde las etiquetas gramaticales eran ambiguas. Estas inconsistencias se documentaron y, cuando fue posible, se corrigieron mediante reglas automáticas o manuales para garantizar la calidad de los datos antes del entrenamiento del clasificador.

Estas tareas iniciales se implementaron en un entorno de Jupyter Notebook, utilizando las herramientas de NLTK para acceder al corpus, procesar los datos y generar visualizaciones preliminares (como histogramas de distribución de sentidos). Los resultados de este análisis sientan las bases para las etapas posteriores de preprocesamiento, extracción de características y entrenamiento de modelos, asegurando una comprensión clara de la estructura y calidad del corpus.

2. Extracción de Características

En esta etapa del laboratorio, se desarrolló el proceso de extracción de características lingüísticas a partir del corpus Senseval 2, utilizando la biblioteca NLTK 3.3 en Python 3.9.13 dentro de un entorno de Jupyter Notebook. El objetivo fue generar representaciones vectoriales robustas que permitieran al clasificador Naive Bayes distinguir los sentidos de las palabras ambiguas "hard" y "serve" en función de su contexto. Las siguientes tareas se llevaron a cabo para construir y optimizar los conjuntos de características:

- a) **Construcción del vocabulario:** Se creó un vocabulario a partir de las 250 palabras más frecuentes en el corpus, excluyendo elementos no informativos como:
 - a. **Stopwords:** Palabras funcionales de alta frecuencia (ej. "the", "and") que aportan poco valor semántico, utilizando la lista de stopwords de NLTK.
 - b. **Signos de puntuación:** Eliminados mediante tokenización y filtrado para reducir el ruido en los datos.
 - c. **Formas gramaticales de las palabras ambiguas:** Se excluyeron variantes como harder, hardest, served o serving para evitar sesgos en la representación del

contexto. Este vocabulario se utilizó como base para generar vectores de características, asegurando que las palabras seleccionadas fueran representativas y relevantes para la tarea de WSD.

- b) **Generación de vectores de características:** Se implementaron dos enfoques principales para capturar el contexto de las palabras objetivo:
 - a. **Presencia de palabras vecinas (Bag of Words):** Se construyó un modelo de bolsa de palabras considerando una ventana de contexto (por ejemplo, ± 5 palabras alrededor de la palabra ambigua). Cada instancia se representó como un vector binario que indica la presencia o ausencia de cada palabra del vocabulario en el contexto, proporcionando una representación simple pero efectiva del entorno léxico.
 - b. **Colocaciones:** Se extrajeron bigramas (pares de palabras consecutivas) previos y posteriores a la palabra ambigua (ej. "serve food" o "hard surface"). Estos bigramas se seleccionaron según su frecuencia y relevancia semántica, utilizando métricas como la información mutua puntual (Pointwise Mutual Information, PMI) para identificar combinaciones significativas que pudieran diferenciar los sentidos.
- c) **Propuesta de un criterio alternativo:** Como enfoque complementario, se propuso el uso de etiquetas gramaticales (POS tags) del contexto como características adicionales. Se etiquetaron las palabras en la ventana de contexto utilizando el POS tagger de NLTK (basado en el conjunto de etiquetas de Penn Treebank), generando un vector que captura la estructura gramatical del entorno (ej. sustantivo, verbo, adjetivo). Este enfoque permite incorporar información morfosintáctica, que puede ser crucial para distinguir sentidos en casos donde el contexto léxico por sí solo es insuficiente (por ejemplo, "serve" como verbo vs. sustantivo en deportes).

Estas tareas se implementaron con un enfoque modular, asegurando que los vectores de características fueran compatibles con el clasificador Naive Bayes. Los conjuntos de características se evaluaron individualmente y en combinación para determinar su impacto en el rendimiento del modelo, utilizando métricas como precisión y F1-score. Este proceso no solo permitió optimizar la representación de los datos, sino también explorar la influencia de diferentes tipos de información lingüística en la desambiguación del sentido de las palabras.

3. Entrenamiento y Evaluación

En esta etapa del laboratorio, se desarrolló el entrenamiento y la evaluación de clasificadores Naive Bayes para la desambiguación del sentido de las palabras "hard" y "serve", utilizando el corpus Senseval 2 y la biblioteca NLTK 3.3 en Python 3.9.13 dentro de un entorno de Jupyter Notebook. El objetivo fue comparar el rendimiento de diferentes conjuntos de características y analizar los resultados para identificar fortalezas y limitaciones en la clasificación de sentidos. Las tareas realizadas se describen a continuación:

- a) Entrenamiento de clasificadores Naive Bayes: Se construyeron cuatro clasificadores independientes, dos para cada palabra objetivo, utilizando diferentes conjuntos de características:
 - a. Para "hard":
 - i. Modelo 1: Palabras vecinas (Bag of Words): Se utilizó un vector binario basado en la presencia de palabras en una ventana de contexto (± 5 palabras) extraídas del vocabulario de las 250 palabras más frecuentes (excluyendo stopwords, signos de puntuación y formas gramaticales de "hard").

- ii. Modelo 2: Colocaciones: Se emplearon bigramas previos y posteriores a la palabra, seleccionados por su relevancia semántica (calculada mediante información mutua puntual).
- b. Para "serve":
 - i. Modelo 1: Palabras vecinas (Bag of Words): Similar al modelo de "hard", utilizando una ventana de contexto para capturar el entorno léxico.
 - ii. Modelo 2: Colocaciones: Se extrajeron bigramas relevantes para "serve", considerando su rol como verbo o sustantivo en diferentes contextos (ej. "serve food" o "tennis serve"). Los datos se dividieron en conjuntos de entrenamiento (80%) y prueba (20%), utilizando la función de partición de NLTK para garantizar una evaluación robusta. Los clasificadores se entrenaron asumiendo independencia condicional entre las características, característica inherente al modelo Naive Bayes.
- b) Evaluación del rendimiento: El desempeño de cada clasificador se evaluó utilizando dos métricas principales:
 - a. Exactitud (accuracy): Proporción de instancias correctamente clasificadas respecto al total de instancias en el conjunto de prueba.
 - b. Matriz de confusión: Representación tabular que detalla las predicciones correctas e incorrectas para cada sentido, permitiendo identificar patrones de error (ej. confusión entre sentidos específicos). Las métricas se calcularon mediante las herramientas de evaluación de NLTK, y los resultados se visualizaron en Jupyter Notebook para facilitar su interpretación.
- c) Resultados obtenidos:
 - a. Para "hard":
 - i. Palabras vecinas: Exactitud de 0.7891, indicando un rendimiento aceptable, pero con margen de mejora, posiblemente debido a la simplicidad del modelo de bolsa de palabras.
 - ii. Colocaciones: Exactitud de 0.8818, mostrando una mejora significativa al capturar relaciones semánticas más específicas mediante bigramas.
 - b. Para "serve":
 - i. Palabras vecinas: Exactitud de 0.6713, reflejando un desafío mayor debido a la mayor cantidad de sentidos (cuatro) y su variabilidad contextual.
 - ii. Colocaciones: Exactitud de 0.7282, con un mejor desempeño al incorporar información más específica del contexto inmediato. Los resultados sugieren que las colocaciones son más efectivas que las palabras vecinas para ambas palabras, probablemente porque capturan patrones semánticos más distintivos.
- d) Análisis de errores: Se identificaron y listaron las instancias clasificadas incorrectamente para cada clasificador, cuando las había. Este análisis incluyó:
 - a. Revisión de los contextos donde ocurrieron los errores, para detectar patrones (ej. sentidos con contextos similares o ambigüedad en las anotaciones del corpus).
 - b. Comparación de las matrices de confusión para identificar qué sentidos eran más propensos a confundirse (ej. HARD1 vs. HARD2, o SERVE10 vs. SERVE12).

- c. Documentación de ejemplos específicos de oraciones mal clasificadas, junto con sus etiquetas predichas y correctas, para facilitar la interpretación de los fallos del modelo.

Este proceso permitió no solo evaluar cuantitativamente el rendimiento de los clasificadores, sino también comprender las limitaciones de los enfoques utilizados, como la sensibilidad del modelo Naive Bayes a desbalances en la distribución de sentidos o la falta de características semánticas más avanzadas (ej. embeddings de palabras). Los resultados y el análisis de errores proporcionan una base sólida para proponer mejoras, como la incorporación de etiquetas POS o la experimentación con otros algoritmos de aprendizaje automático.

4. Reflexión Crítica

En el análisis de los resultados obtenidos en el laboratorio de desambiguación del sentido de las palabras (WSD), se determinó que las características basadas en colocaciones (bigramas previos y posteriores a las palabras objetivo "hard" y "serve") ofrecieron un rendimiento superior en comparación con las características de palabras vecinas (modelo de bolsa de palabras). Específicamente, los clasificadores Naive Bayes alcanzaron exactitudes de 0.8818 para "hard" y 0.7282 para "serve" con colocaciones, frente a 0.7891 y 0.6713 con palabras vecinas, respectivamente. Este mejor desempeño se atribuye a la capacidad de las colocaciones para capturar relaciones semánticas más específicas y contextualmente relevantes (por ejemplo, "serve food" para SERVE10 o "hard surface" para HARD1), mientras que el modelo de bolsa de palabras ofrece una representación más general que no considera el orden ni la co-ocurrencia de las palabras.

Observaciones clave:

- a) Dificultad con sentidos menos frecuentes: Se identificó que los sentidos con menor representación en el corpus Senseval 2, como SERVE6 (asociado a la acción de "servir" en deportes, ej. "tennis serve"), presentaron mayores tasas de error en la clasificación. Esto se explica por:
 - a. Desbalance de datos: La escasa cantidad de instancias de SERVE6 limita la capacidad del clasificador Naive Bayes para aprender patrones distintivos, reduciendo la sensibilidad para este sentido.
 - b. Similitud contextual: Algunos sentidos, como SERVE6 y SERVE12 (atender o asistir), comparten contextos léxicos y gramaticales similares, lo que genera confusión en el modelo, como se evidenció en las matrices de confusión analizadas. Estas observaciones resaltan la necesidad de abordar el desbalance de clases y mejorar la captura de características distintivas para sentidos menos frecuentes.
- b) Propuestas de mejora: Para optimizar el rendimiento de los clasificadores y superar las limitaciones identificadas, se plantearon las siguientes estrategias:
 - a. Adopción de modelos más complejos: Implementar algoritmos de aprendizaje automático más avanzados, como Support Vector Machines (SVM), Random Forest o modelos basados en transformers (ej. BERT). Estos modelos pueden modelar relaciones no lineales y dependencias contextuales complejas, superando las restricciones del supuesto de independencia condicional del algoritmo Naive Bayes.
 - b. Incorporación de embeddings semánticos: Utilizar representaciones vectoriales de palabras basadas en modelos preentrenados, como Word2Vec o BERT, para capturar información semántica más rica. Por ejemplo, los embeddings contextuales de BERT podrían mejorar la diferenciación de sentidos como SERVE6 al considerar el significado global de la oración.
 - c. Enriquecimiento de características lingüísticas: Integrar técnicas de preprocesamiento avanzadas, como lematización (para normalizar formas

gramaticales, ej. reducir "serving" a "serve") y etiquetas gramaticales (POS tags), utilizando el POS tagger de NLTK basado en el conjunto de etiquetas de Penn Treebank. Estas características morfosintácticas pueden ayudar a distinguir sentidos basados en roles gramaticales (ej. "serve" como verbo vs. sustantivo).

El análisis y las propuestas se desarrollaron utilizando Python 3.9.13, NLTK 3.3 y Jupyter Notebook, con visualizaciones de matrices de confusión generadas para facilitar la interpretación de los errores de clasificación y orientar las mejoras futuras.

5. Explicación De Todo El Código

A continuación, procedemos con la explicación de todo el código implementado en Jupiter Notebook:

Parte 1: Análisis del Corpus

a) Cargar el corpus Senseval 2

```
[1]: import nltk
    nltk.download('senseval')
    from nltk.corpus import senseval

    # Mostrar las palabras ambiguas disponibles en el corpus
    print("Palabras disponibles en el corpus Senseval 2:")
    print(senseval.fileids())

[nltk_data] Downloading package senseval to
[nltk_data] C:\Users\gran\AppData\Roaming\nltk_data...
[nltk_data] Package senseval is already up-to-date!
Palabras disponibles en el corpus Senseval 2:
['hard.pos', 'interest.pos', 'line.pos', 'serve.pos']
```

Este código representa el punto de partida para el laboratorio de desambiguación del sentido de las palabras. Su propósito es inicializar el acceso al corpus Senseval 2 y confirmar las palabras ambiguas disponibles, asegurando que los datos necesarios para analizar "hard" y "serve" estén presentes. Este paso es crítico porque:

Verifica que el corpus esté correctamente instalado y accesible.

Identifica los archivos relevantes (hard.pos y serve.pos) que contienen las instancias etiquetadas necesarias para las tareas posteriores, como el análisis de sentidos, la extracción de características y el entrenamiento de clasificadores Naive Bayes.

Proporciona una visión general de las palabras ambiguas disponibles, lo que permite al usuario confirmar que el corpus cubre las palabras objetivo del laboratorio.

A continuación, su explicación línea por línea:

Línea 1: import nltk

Importa la biblioteca NLTK, que proporciona herramientas para el procesamiento del lenguaje natural, incluyendo acceso a corpus lingüísticos, funciones de tokenización, etiquetado gramatical (POS tagging) y algoritmos de aprendizaje automático para tareas como WSD.

Se aclara que NLTK es la biblioteca principal utilizada en el laboratorio para manejar el corpus Senseval 2 y realizar tareas como la extracción de instancias, preprocesamiento y entrenamiento de clasificadores. La versión implícita en el laboratorio es NLTK 3.3, como se mencionó en el contexto del proyecto.

Línea 2: nltk.download('senseval')

Descarga el corpus Senseval 2 desde los servidores de NLTK y lo almacena localmente en el directorio de datos del usuario (en este caso, C:\Users\gran\AppData\Roaming\nltk_data\). Si el

corpus ya está descargado, NLTK omite la descarga y muestra un mensaje indicando que el paquete está actualizado.

Salida observada

```
[nltk_data] Downloading package senseval to  
[nltk_data]   C:\Users\gran_\AppData\Roaming\nltk_data...  
[nltk_data]   Package senseval is already up-to-date!
```

Este mensaje confirma que el corpus Senseval 2 ya está instalado en el sistema, por lo que no se realiza una nueva descarga. Esto es importante para asegurar que los datos estén disponibles antes de intentar acceder a ellos.

Contexto

El corpus Senseval 2 contiene instancias etiquetadas de palabras ambiguas en inglés (como "hard" y "serve"), donde cada instancia incluye una palabra objetivo, su contexto (oración o fragmento) y una etiqueta que indica su sentido específico. Este paso garantiza que el corpus esté accesible para las tareas posteriores del laboratorio.

Línea 3: `from nltk.corpus import senseval`

Importa específicamente el módulo senseval desde el paquete nltk.corpus, que proporciona acceso directo al corpus Senseval 2. Este módulo permite cargar las instancias del corpus, consultar las palabras ambiguas disponibles y extraer información como los sentidos, contextos y etiquetas gramaticales.

Se aclara que este paso es fundamental para trabajar con los datos de Senseval 2, ya que el módulo senseval ofrece funciones específicas para acceder a las palabras ambiguas y sus instancias, como se verá en el siguiente paso.

Línea 4: `print("Palabras disponibles en el corpus Senseval 2:")`

Imprime un mensaje en la consola para indicar que se mostrará la lista de palabras ambiguas disponibles en el corpus. Esto mejora la legibilidad de la salida y proporciona contexto al usuario.

Se aclara que este mensaje es un encabezado descriptivo que prepara al usuario para interpretar la información que sigue, es decir, los nombres de los archivos que contienen las instancias de las palabras ambiguas.

Línea 5: `print(senseval.fileids())`

Llama al método fileids() del objeto senseval, que devuelve una lista de los nombres de los archivos que componen el corpus Senseval 2. Cada archivo corresponde a una palabra ambigua específica y contiene las instancias etiquetadas para esa palabra.

Salida observada

```
Palabras disponibles en el corpus Senseval 2:  
['hard.pos', 'interest.pos', 'line.pos', 'serve.pos']
```

Esta salida indica que el corpus Senseval 2 incluye datos para cuatro palabras ambiguas: "hard", "interest", "line" y "serve". Cada archivo (ej. hard.pos) contiene las instancias de la palabra correspondiente, con información sobre su contexto, sentido etiquetado y etiquetas gramaticales (POS tags).

Contexto

En el laboratorio, las palabras de interés son "hard" y "serve", por lo que los archivos relevantes son hard.pos y serve.pos. El método fileids() es útil para explorar el contenido del corpus antes de realizar tareas como la extracción de instancias o el análisis de sentidos.

Parte 2: Contar los sentidos disponibles y la cantidad de instancias por cada sentido

```
[2]: from collections import Counter

# Obtener las instancias de las palabras
hard_instances = senseval.instances('hard.pos')
serve_instances = senseval.instances('serve.pos')

# Contar los sentidos para cada palabra
hard_senses = Counter()
for inst in hard_instances:
    hard_senses.update(inst.senses)

serve_senses = Counter()
for inst in serve_instances:
    serve_senses.update(inst.senses)

print("Sentidos para la palabra 'hard':")
for sense, count in hard_senses.items():
    print(f"{sense}: {count} instancias")

print("\nSentidos para la palabra 'serve':")
for sense, count in serve_senses.items():
    print(f"{sense}: {count} instancias")
```

```
Sentidos para la palabra 'hard':
HARD1: 3455 instancias
HARD2: 502 instancias
HARD3: 376 instancias
```

```
Sentidos para la palabra 'serve':
SERVE10: 1814 instancias
SERVE12: 1272 instancias
SERVE2: 853 instancias
SERVE6: 439 instancias
```

Este código presentado utiliza Python y la biblioteca NLTK (Natural Language Toolkit), junto con el módulo Counter de la biblioteca estándar collections, para analizar el corpus Senseval 2. Su propósito es cargar las instancias de las palabras ambiguas "hard" y "serve" desde los archivos hard.pos y serve.pos, contar la frecuencia de cada sentido asociado a estas palabras, y mostrar los resultados. Este paso es fundamental en el laboratorio para entender la distribución de los sentidos en el corpus, lo cual es clave para tareas posteriores como el entrenamiento de clasificadores Naive Bayes. A continuación, se desglosa cada parte del código:

Línea 1: from collections import Counter

Importa la clase Counter del módulo collections, una herramienta de Python diseñada para contar elementos en una colección iterable (como una lista o tupla). Counter crea un diccionario donde las claves son los elementos únicos y los valores son sus frecuencias.

Se aclara que, en este caso, Counter se utiliza para contar cuántas veces aparece cada sentido (etiqueta de sentido) de las palabras "hard" y "serve" en las instancias del corpus. Esto es útil para analizar la distribución de sentidos, identificar posibles desbalances y preparar los datos para el entrenamiento de clasificadores.

Líneas 3-4: Carga de instancias

Utiliza el método senseval.instances() del módulo senseval de NLTK para cargar todas las instancias etiquetadas de las palabras "hard" y "serve" desde los archivos hard.pos y serve.pos, respectivamente.

Código:

```
# Obtener las instancias de las palabras
hard_instances = senseval.instances('hard.pos')
serve_instances = senseval.instances('serve.pos')
```

Detalles:

senseval.instances('hard.pos') devuelve una lista de objetos SensevalInstance, donde cada objeto representa una instancia de la palabra "hard" en el corpus. Cada instancia incluye:

- El contexto (generalmente una oración o fragmento donde aparece la palabra).
- La etiqueta de sentido (ej. HARD1, HARD2, HARD3).
- Etiquetas gramaticales (POS tags) para la palabra y su contexto.
- La posición de la palabra objetivo en el contexto.

De manera similar, senseval.instances('serve.pos') carga las instancias de "serve".

Contexto:

Este paso es esencial para acceder a los datos del corpus Senseval 2, que contiene ejemplos etiquetados para la tarea de WSD. Los archivos hard.pos y serve.pos fueron identificados previamente en el laboratorio (como se vio en el código anterior) como parte del corpus.

Líneas 6-9: Conteo de sentidos para "hard"

Crea un objeto Counter vacío (hard_senses) y lo utiliza para contar la frecuencia de cada sentido de la palabra "hard" en el corpus. Y luego similar al paso anterior, se crea un objeto Counter (serve_senses) para contar la frecuencia de cada sentido de la palabra "serve" en el corpus.

Código:

```
# Contar los sentidos para cada palabra
hard_senses = Counter()
for inst in hard_instances:
    hard_senses.update(inst.senses)
```

Detalles:

- Counter() inicializa un contador vacío.
- El bucle for inst in hard_instances itera sobre cada instancia de "hard".
- inst.senses accede a la lista de sentidos etiquetados para la instancia. En el corpus Senseval 2, cada instancia tiene una lista de sentidos (generalmente un solo sentido por instancia, pero el atributo es una lista para permitir flexibilidad).
- hard_senses.update(inst.senses) incrementa el conteo de cada sentido encontrado en la instancia, acumulando las frecuencias en el objeto Counter.

Contexto:

Este paso permite cuantificar cuántas instancias corresponden a cada sentido de "hard" (ej. HARD1, HARD2, HARD3), proporcionando una visión inicial de la distribución de sentidos.

Líneas 11-14: Conteo de sentidos para "serve"

Código:

```
serve_senses = Counter()
for inst in serve_instances:
    serve_senses.update(inst.senses)
```

Detalles:

El proceso es idéntico al de "hard", pero aplicado a las instancias de "serve", que tienen sentidos como SERVE10, SERVE12, SERVE2 y SERVE6.

Contexto:

Este conteo es crucial para identificar la distribución de sentidos de "serve", especialmente porque tiene más sentidos (cuatro) que "hard" (tres), lo que puede implicar mayor complejidad en la desambiguación.

Líneas 16-19: Impresión de resultados para "hard"

Imprime la distribución de sentidos para la palabra "hard", mostrando cada sentido y su número de instancias.

Código:

```
print("Sentidos para la palabra 'hard':")
for sense, count in hard_senses.items():
    print(f"{sense}: {count} instancias")
```

Salida observada:

```
Sentidos para la palabra 'hard':
HARD1: 3455 instancias
HARD2: 502 instancias
HARD3: 376 instancias
```

Interpretación:

- HARD1: 3455 instancias, el sentido más frecuente (probablemente asociado a "dureza física", ej. "superficie dura").
- HARD2: 502 instancias, menos frecuente (posiblemente "dificultad", ej. "tarea difícil").
- HARD3: 376 instancias, el menos frecuente (probablemente "esfuerzo o severidad", ej. "trabajar duro").

Esta distribución muestra un desbalance significativo, con HARD1 dominando el corpus, lo que puede influir en el rendimiento de los clasificadores, ya que el modelo Naive Bayes podría sesgarse hacia el sentido más frecuente.

Líneas 21-24: Impresión de resultados para "serve"

Imprime la distribución de sentidos para la palabra "serve", mostrando cada sentido y su número de instancias.

Código:

```
print("\nSentidos para la palabra 'serve':")
for sense, count in serve_senses.items():
    print(f"{sense}: {count} instancias")
```

Salida observada:

```
Sentidos para la palabra 'serve':
SERVE10: 1814 instancias
SERVE12: 1272 instancias
SERVE2: 853 instancias
SERVE6: 439 instancias
```

Interpretación:

- SERVE10: 1814 instancias, el sentido más frecuente (probablemente "servir comida o bebida", ej. "servir la cena").
- SERVE12: 1272 instancias (posiblemente "atender o asistir", ej. "servir a un cliente").
- SERVE2: 853 instancias (probablemente "cumplir una función", ej. "servir como ejemplo").
- SERVE6: 439 instancias, el menos frecuente (probablemente "acción en deportes", ej. "servir en tenis").

Similar a "hard", hay un desbalance en la distribución, con SERVE6 teniendo significativamente menos instancias, lo que, como se mencionó en el laboratorio, dificulta su desambiguación.

Parte 3: Identificación de formas gramaticales

```
[4]: from collections import Counter

def formas_gramaticales(instancias, palabra_base):
    formas = Counter()
    for inst in instancias:
        context = list(inst.context) # aseguramos que es una lista de tuplas
        for elemento in context:
            if isinstance(elemento, tuple) and len(elemento) == 2:
                word, tag = elemento
                if palabra_base in word.lower():
                    formas[(word.lower(), tag)] += 1
    return formas

print("Formas gramaticales de 'hard':")
formas_hard = formas_gramaticales(hard_instancias, 'hard')
for forma, count in formas_hard.items():
    print(f"{forma}: {count} veces")

print("\nFormas gramaticales de 'serve':")
formas_serve = formas_gramaticales(serve_instancias, 'serve')
for forma, count in formas_serve.items():
    print(f"{forma}: {count} veces")
```

Formas gramaticales de 'hard':

```
('hard', 'JJ'): 3706 veces
('harder', 'JJ'): 501 veces
('hardest', 'JJ'): 140 veces
('die-hard', 'JJ'): 1 veces
('harder', 'NNP'): 5 veces
('hardaway', 'NNP'): 1 veces
('harder', 'JJR'): 1 veces
```

Formas gramaticales de 'serve':

```
('serve', 'VB'): 1983 veces
('preserve', 'VB'): 2 veces
('served', 'VBD'): 1426 veces
('serves', 'VBZ'): 706 veces
('served', 'VBN'): 68 veces
('serve', 'VBP'): 14 veces
('reserved', 'VBN'): 12 veces
('reserve', 'NNP'): 8 veces
('server', 'NN'): 2 veces
('reserve', 'NN'): 7 veces
('observed', 'VBN'): 2 veces
('deserved', 'VBD'): 2 veces
('32serves', 'NNS'): 1 veces
('observe', 'VB'): 3 veces
('reserves', 'NNS'): 4 veces
('deserve', 'VBP'): 3 veces
('observer', 'NN'): 2 veces
('observers', 'NNS'): 2 veces
('observed', 'VBD'): 2 veces
```

Este código define una función que analiza las formas gramaticales de "hard" y "serve" en el corpus Senseval 2, contando las frecuencias de cada combinación de forma léxica y etiqueta POS. La salida revela:

- "hard": Principalmente adjetivo (JJ, JJR, JJS), con errores de etiquetado (ej. "harder" como JJ) y formas raras (ej. "hardaway").
- "serve": Predominantemente verbal (VB, VBD, VBZ, VBN, VBP), pero con formas irrelevantes (ej. "preserve", "observe") y errores como "32serves".

Este análisis es clave para el laboratorio, ya que identifica la diversidad gramatical, detecta inconsistencias en el corpus, y proporciona información para enriquecer las características de los clasificadores Naive Bayes. Las limitaciones observadas (etiquetado incorrecto, formas irrelevantes) destacan la necesidad de preprocesamiento adicional. A continuación, la explicación línea por línea del código:

Línea 1: `from collections import Counter`

Importa la clase Counter del módulo collections, que permite contar la frecuencia de elementos en una colección iterable. En este caso, Counter se usa para registrar cuántas veces aparece cada combinación de forma léxica y etiqueta gramatical (POS tag) asociada a las palabras "hard" y "serve". Es importante aclarar que este módulo es ideal para tareas de análisis exploratorio, ya que simplifica el conteo de ocurrencias y produce un diccionario con claves (formas gramaticales) y valores (frecuencias).

Líneas 3-10: Definición de la función `formas_gramaticales`

Define una función reusable que toma como entrada una lista de instancias de Senseval 2 (objetos SensevalInstance) y una palabra base (ej. "hard" o "serve"), y devuelve un objeto Counter con las frecuencias de las formas léxicas (palabras) y sus etiquetas gramaticales (POS tags) que contienen la palabra base en minúsculas.

Código:

```
def formas_gramaticales(instances, palabra_base):
    formas = Counter()
    for inst in instances:
        context = list(inst.context) # aseguramos que es una lista de tuplas
        for elemento in context:
            if isinstance(elemento, tuple) and len(elemento) == 2:
                word, tag = elemento
                if palabra_base in word.lower():
                    formas[(word.lower(), tag)] += 1
    return formas
```

Parámetros:

- instances: Lista de instancias del corpus (ej. hard_instances o serve_instances), donde cada instancia contiene el contexto de la palabra objetivo, su sentido y etiquetas POS.
- palabra_base: Cadena que representa la palabra base (ej. "hard" o "serve") usada para buscar formas relacionadas.
- formas = Counter(): Inicializa un objeto Counter vacío para almacenar las combinaciones de forma léxica y POS tag.
- for inst in instances: Itera sobre cada instancia del corpus.

- `context = list(inst.context)`: Convierte el atributo `context` de la instancia (que contiene el texto circundante de la palabra objetivo) en una lista de tuplas. Cada tupla tiene la forma (palabra, POS_tag).
- `for elemento in context`: Itera sobre los elementos del contexto.
- `if isinstance(elemento, tuple) and len(elemento) == 2`: Verifica que cada elemento sea una tupla de dos elementos (palabra y POS tag), asegurando que el formato sea correcto.
- `word, tag = elemento`: Desempaqueta la tupla en la palabra y su etiqueta gramatical.
- `if palabra_base in word.lower()`: Comprueba si la palabra base (en minúsculas) está contenida en la palabra actual (en minúsculas), permitiendo capturar formas derivadas como "harder", "served", etc.
- `formas[(word.lower(), tag)] += 1`: Incrementa el conteo de la combinación (palabra, POS_tag) en el objeto Counter.
- `return formas`: Devuelve el objeto Counter con las frecuencias de todas las formas gramaticales encontradas.

Contexto:

Esta función es clave para identificar las variaciones morfológicas de las palabras objetivo (ej. "hard", "harder", "served") y sus roles gramaticales (ej. adjetivo, verbo), lo que ayuda a caracterizar el corpus y detectar posibles inconsistencias (como formas inesperadas o etiquetas erróneas).

Líneas 12-15: Análisis de formas gramaticales para "hard"

Aplica la función `formas_gramaticales` a las instancias de "hard" (`hard_instances`, cargadas previamente con `senseval.instances('hard.pos')`) y muestra las formas léxicas y sus etiquetas POS junto con sus frecuencias.

Código:

```
print("Formas gramaticales de 'hard':")
formas_hard = formas_gramaticales(hard_instances, 'hard')
for forma, count in formas_hard.items():
    print(f"{forma}: {count} veces")
```

Salida observada:

```
Formas gramaticales de 'hard':
('hard', 'JJ'): 3706 veces
('harder', 'JJ'): 501 veces
('hardest', 'JJ'): 140 veces
('die-hard', 'JJ'): 1 veces
('harder', 'NNP'): 5 veces
('hardaway', 'NNP'): 1 veces
('harder', 'JJR'): 1 veces
```

Interpretación:

- ('hard', 'JJ'): 3706 ocurrencias como adjetivo (JJ), la forma más común, representando el uso estándar de "hard" (ej. "superficie dura").
- ('harder', 'JJ'): 501 ocurrencias como adjetivo, probablemente un error en el etiquetado, ya que "harder" debería ser JJR (comparativo).
- ('hardest', 'JJ'): 140 ocurrencias como adjetivo, también un posible error, ya que "hardest" debería ser JJS (superlativo).
- ('die-hard', 'JJ'): 1 ocurrencia como adjetivo compuesto, un caso raro pero válido.
- ('harder', 'NNP'): 5 ocurrencias como nombre propio, posiblemente un error o un caso contextual específico (ej. un nombre propio como "Harder").

- ('hardaway', 'NNP'): 1 ocurrencia como nombre propio, probablemente un error o un nombre específico en el contexto.
- ('harder', 'JJR'): 1 ocurrencia como comparativo, que parece ser el etiquetado correcto para "harder".

Líneas 17-20: Análisis de formas gramaticales para "serve"

Aplica la función `formas_gramaticales` a las instancias de "serve" (`serve_instances`, cargadas con `senseval.instances('serve.pos')`) y muestra las formas léxicas y sus etiquetas POS junto con sus frecuencias.

Código:

```
print("\nFormas gramaticales de 'serve':")
formas_serve = formas_gramaticales(serve_instances, 'serve')
for forma, count in formas_serve.items():
    print(f"{forma}: {count} veces")
```

Salida observada:

```
Formas gramaticales de 'serve':
('serve', 'VB'): 1983 veces
('preserve', 'VB'): 2 veces
('served', 'VBD'): 1426 veces
('serves', 'VBZ'): 706 veces
('served', 'VBN'): 68 veces
('serve', 'VBP'): 14 veces
('reserved', 'VBN'): 12 veces
('reserve', 'NNP'): 8 veces
('server', 'NN'): 2 veces
('reserve', 'NN'): 7 veces
('observed', 'VBN'): 2 veces
('deserved', 'VBD'): 2 veces
('32serves', 'NNS'): 1 veces
('observe', 'VB'): 3 veces
('reserves', 'NNS'): 4 veces
('deserve', 'VBP'): 3 veces
('observer', 'NN'): 2 veces
('observers', 'NNS'): 2 veces
('observed', 'VBD'): 2 veces
```

Interpretación:

- ('serve', 'VB'): 1983 ocurrencias como verbo en forma base (infinitivo o imperativo).
- ('served', 'VBD'): 1426 ocurrencias como verbo en pasado simple.
- ('serves', 'VBZ'): 706 ocurrencias como verbo en tercera persona singular.
- ('served', 'VBN'): 68 ocurrencias como participio pasado.
- ('serve', 'VBP'): 14 ocurrencias como verbo en presente (no tercera persona).
- Formas relacionadas pero inesperadas:
 - ('preserve', 'VB'), ('reserved', 'VBN'), ('reserve', 'NNP'), ('reserve', 'NN'), ('reserves', 'NNS'):
- Formas derivadas de la raíz "serv-" (conservar, reservar):
 - ('observe', 'VB'), ('observed', 'VBN'), ('observed', 'VBD'), ('observer', 'NN'), ('observers', 'NNS')
- Palabras relacionadas con "observar", incluidas debido a la búsqueda con `in word.lower()`, lo que captura cualquier palabra que contenga "serve".
 - ('deserve', 'VBP'), ('deserved', 'VBD')

- Palabras relacionadas con "merecer", también incluidas por la búsqueda amplia.
 - ('server', 'NN'): 2 ocurrencias como sustantivo (ej. "servidor"), que podría ser relevante en algunos contextos (ej. tecnología).
 - ('32serves', 'NNS'): 1 ocurrencia, claramente un error o ruido en el corpus, probablemente un problema de tokenización.

Parte 4: Validación del formato de las instancias

```
[5]: def verificar_formato(instancias, palabra):
    errores = []
    for i, inst in enumerate(instancias):
        if not hasattr(inst, 'word') or not hasattr(inst, 'context') or not hasattr(inst, 'senses') or not hasattr(inst, 'position'):
            errores.append((i, "Faltan campos"))
            continue
        for elemento in inst.context:
            if not (isinstance(elemento, tuple) and len(elemento) == 2):
                errores.append((i, f"Contexto mal formateado: {elemento}"))
                break
    print(f"Instancias mal formateadas para '{palabra}': {len(errores)} encontradas.")
    for i, error in errores[:5]: # muestra solo los primeros 5
        print(f"Instancia {i}: {error}")
    return errores

errores_hard = verificar_formato(hard_instancias, 'hard')
errores_serve = verificar_formato(serve_instancias, 'serve')
```

Instancias mal formateadas para 'hard': 19 encontradas.
 Instancia 21: Contexto mal formateado: FRASL
 Instancia 110: Contexto mal formateado: FRASL
 Instancia 196: Contexto mal formateado: FRASL
 Instancia 892: Contexto mal formateado: FRASL
 Instancia 999: Contexto mal formateado: FRASL
 Instancias mal formateadas para 'serve': 76 encontradas.
 Instancia 18: Contexto mal formateado: FRASL
 Instancia 24: Contexto mal formateado: FRASL
 Instancia 133: Contexto mal formateado: FRASL
 Instancia 150: Contexto mal formateado: FRASL
 Instancia 151: Contexto mal formateado: FRASL

El código define una función que verifica el formato de las instancias de "hard" y "serve" en el corpus Senseval 2, detectando errores en los campos esenciales o en el formato del contexto. La salida muestra:

- "hard": 19 instancias mal formateadas (0.44% del total), todas con "FRASL" en el contexto.
- "serve": 76 instancias mal formateadas (1.74% del total), también con "FRASL".

Este análisis es crucial para la limpieza de datos en el laboratorio, confirmando la presencia de inconsistencias en el corpus (como se mencionó previamente) y destacando la necesidad de preprocesamiento adicional para garantizar la calidad de los datos antes de entrenar los clasificadores Naive Bayes. A continuación, la explicación línea por línea del código:

Líneas 1-11: Definición de la función verificar_formato

Define una función reusable que toma como entrada una lista de instancias de Senseval 2 (objetos SensevalInstance) y el nombre de la palabra objetivo (ej. "hard" o "serve"), y devuelve una lista de errores encontrados en las instancias, identificando problemas en los campos requeridos o en el formato del contexto.

Código:

```
[5]: def verificar_formato(instancias, palabra):
    errores = []
    for i, inst in enumerate(instancias):
        if not hasattr(inst, 'word') or not hasattr(inst, 'context') or not hasattr(inst, 'senses') or not hasattr(inst, 'position'):
            errores.append((i, "Faltan campos"))
            continue
        for elemento in inst.context:
            if not (isinstance(elemento, tuple) and len(elemento) == 2):
                errores.append((i, f"Contexto mal formateado: {elemento}"))
                break
```

Parámetros:

- **instances:** Lista de instancias del corpus (ej. `hard_instances` o `serve_instances`), cargadas previamente con `senseval.instances('hard.pos')` o `senseval.instances('serve.pos')`.
- **palabra:** Cadena que indica la palabra objetivo, usada solo para personalizar los mensajes de salida.
- **errores = []:** Inicializa una lista vacía para almacenar los errores encontrados, donde cada error es una tupla (índice, descripción).
- **for i, inst in enumerate(instances):** Itera sobre las instancias, utilizando `enumerate` para obtener el índice de cada instancia (útil para identificarlas en los mensajes de error).
- **Verificación de campos:**
 - `if not hasattr(inst, 'word') or not hasattr(inst, 'context') or not hasattr(inst, 'senses') or not hasattr(inst, 'position'):` Comprueba si la instancia tiene los atributos esenciales de un objeto `SensevalInstance`:
 - **word:** La palabra objetivo (ej. "hard" o "serve").
 - **context:** El contexto de la palabra (lista de tuplas (palabra, POS_tag)).
 - **senses:** La lista de sentidos etiquetados.
 - **position:** El índice de la palabra objetivo en el contexto.
 - Si falta algún atributo, se agrega una tupla (i, "Faltan campos") a la lista de errores y se pasa a la siguiente instancia (`continue`).
- **Verificación del contexto:**
 - `for elemento in inst.context:` Itera sobre los elementos del contexto de la instancia.
 - `if not (isinstance(elemento, tuple) and len(elemento) == 2):` Comprueba que cada elemento del contexto sea una tupla de exactamente dos elementos (palabra y POS tag). Si no cumple esta condición (ej. si es una cadena en lugar de una tupla), se considera un error.
 - `errores.append((i, f"Contexto mal formateado: {elemento}")):` Agrega una tupla con el índice de la instancia y una descripción del error, incluyendo el elemento problemático.
 - `break:` Detiene la verificación del contexto de la instancia actual tras encontrar el primer error, para evitar registrar múltiples errores por la misma instancia.
 - `return errores:` Devuelve la lista de errores encontrados.

Contexto:

Esta función es parte de la etapa de limpieza de datos del laboratorio, donde se busca identificar instancias mal formateadas que puedan afectar la extracción de características o el entrenamiento de los clasificadores Naive Bayes. La detección de errores es crítica para garantizar la calidad de los datos.

Líneas 13-17: Impresión de errores

Propósito: Imprime un resumen de los errores encontrados para la palabra dada, mostrando el número total de instancias mal formateadas y los detalles de hasta los primeros cinco errores.

Código:

```
print(f"Instancias mal formateadas para '{palabra}': {len(errores)} encontradas.")
for i, error in errores[:5]: # muestra solo los primeros 5
    print(f"Instancia {i}: {error}")
return errores
```

Detalles:

- `print(f"Instancias mal formateadas para '{palabra}': {lenerrores}} encontradas.")`: Muestra el nombre de la palabra y el total de errores encontrados.
- `for i, error in errores[:5]`: Itera sobre los primeros cinco elementos de la lista de errores (si los hay), usando desempaqueo de tuplas para obtener el índice y la descripción del error.
- `print(f"Instancia {i}: {error}")`: Imprime el índice de la instancia y el mensaje de error asociado.
- La limitación a cinco errores (`[:5]`) evita saturar la salida si hay muchos errores, manteniendo la legibilidad.

Contexto:

Esta impresión es útil para inspeccionar rápidamente los problemas en los datos y decidir cómo manejarlos (ej. eliminar instancias, corregirlas manualmente o ignorarlas).

Líneas 19-20: Ejecución de la función

Aplica la función `verificar_formato` a las instancias de "hard" (`hard_instances`) y "serve" (`serve_instances`), previamente cargadas con `senseval.instances('hard.pos')` y `senseval.instances('serve.pos')`, y almacena los errores encontrados en las variables `errores_hard` y `errores_serve`.

Código:

```
errores_hard = verificar_formato(hard_instances, 'hard')
errores_serve = verificar_formato(serve_instances, 'serve')
```

Contexto:

Estas líneas ejecutan el análisis de formato para las dos palabras objetivo del laboratorio, integrándose con las tareas de limpieza de datos mencionadas previamente.

Salida observada:

Para "hard":

```
Instancias mal formateadas para 'hard': 19 encontradas.
Instancia 21: Contexto mal formateado: FRASL
Instancia 110: Contexto mal formateado: FRASL
Instancia 196: Contexto mal formateado: FRASL
Instancia 892: Contexto mal formateado: FRASL
Instancia 999: Contexto mal formateado: FRASL
```

Para "serve":

```
Instancias mal formateadas para 'serve': 76 encontradas.
Instancia 18: Contexto mal formateado: FRASL
Instancia 24: Contexto mal formateado: FRASL
Instancia 133: Contexto mal formateado: FRASL
Instancia 150: Contexto mal formateado: FRASL
Instancia 151: Contexto mal formateado: FRASL
```

Interpretación:

- "hard": Se encontraron 19 instancias mal formateadas, todas con el mismo error: un elemento en el contexto etiquetado como "FRASL", que no es una tupla (palabra, POS_tag) sino una cadena. FRASL probablemente representa una barra inclinada ("/") mal procesada durante la tokenización del corpus, indicando un error en el formato de los datos.

- "serve": Se encontraron 76 instancias mal formateadas, también debido a la presencia de "FRASL" en el contexto. El mayor número de errores en "serve" (76 vs. 19) puede deberse a la mayor cantidad de instancias totales (4378 para "serve" vs. 4333 para "hard", según el conteo de sentidos previo) o a una mayor incidencia de problemas de tokenización en el archivo serve.pos.

Parte 5: Extracción de características basada en palabras vecinas (Bag of Words) - Filtrado de palabras (limpieza y stopwords)

```
[7]: from nltk.corpus import stopwords
import string
from nltk import FreqDist

# Stopwords y símbolos a eliminar
stop_words = set(stopwords.words('english')).union(set(string.punctuation))
otros_ruidos = set(['"', "'d", "'ll", "'m", "'re", "'s", "'t", "'ve", "--", '000', '1', '2', '3', '4', '5', '6', '8', '10', '15', '30', 'I', 'F', '''])

# Formas gramaticales de "hard" que queremos eliminar
formas_hard_palabras = set([w for (w, _) in formas_hard.keys()])

# Unir todos los filtros
palabras_omitidas = stop_words.union(otros_ruidos).union(formas_hard_palabras)

# Recoger palabras válidas del contexto
tokens_hard = []
for inst in hard_instances:
    for elemento in inst.context:
        if isinstance(elemento, tuple) and len(elemento) == 2:
            word, tag = elemento
            word_lower = word.lower()
            if word_lower not in palabras_omitidas:
                tokens_hard.append(word_lower)

# Calcular las 250 palabras más frecuentes
fdist_hard = FreqDist(tokens_hard)
vocabulario_hard = list(fdist_hard)[:250]

print("Primeras 10 palabras del vocabulario para 'hard':")
print(vocabulario_hard[:10])
```

El código genera un vocabulario de las 250 palabras más frecuentes en los contextos de "hard" en el corpus Senseval 2, filtrando stopwords, signos de puntuación, formas de "hard", y otros términos de ruido. La salida (['may', 'lose', 'popular', 'support', 'someone', 'kill', 'defeat', 'clever', 'white', 'house']) muestra las primeras 10 palabras, que son razonablemente relevantes, pero incluyen algunos términos generales. Este vocabulario es clave para el modelo de bolsa de palabras en el laboratorio. A continuación, la explicación línea por línea del código:

Líneas 1-3: Importaciones

El propósito de este código se basa en lo siguiente:

- stopwords: Importa el módulo de NLTK que proporciona una lista de palabras vacías (stopwords) en inglés, como "the", "and", que suelen carecer de valor semántico en tareas de procesamiento de lenguaje natural.
- string: Importa el módulo estándar de Python que incluye una lista de signos de puntuación (string.punctuation), como ",", ".", "!", usados para filtrar caracteres no deseados.
- FreqDist: Importa la clase FreqDist de NLTK, que calcula la distribución de frecuencias de elementos en una lista, similar a Counter pero optimizada para tareas de procesamiento de texto.

Código:

```
[7]: from nltk.corpus import stopwords
import string
from nltk import FreqDist
```

Contexto:

Estas herramientas son esenciales para el preprocesamiento de los datos del corpus, eliminando términos irrelevantes y generando un vocabulario limpio para la tarea de WSD.

Líneas 5-6: Definición de filtros iniciales

El propósito de este código se basa en lo siguiente:

- `stop_words`: Combina las stopwords en inglés de NLTK (ej. "the", "is") con los signos de puntuación de `string.punctuation` (ej. ",", "."), convirtiéndolos en un conjunto (`set`) para búsquedas eficientes. Esto elimina palabras y caracteres que no aportan información semántica relevante.
- `otros_ruidos`: Define un conjunto adicional de términos considerados ruido, incluyendo:
 - Contracciones y formas verbales abreviadas (ej. "'d", "'ll", "'m").
 - Guiones y números (ej. "--", "000", "1", "2").
 - Palabras comunes o poco informativas (ej. "also", "said", "say", "says", "one").
 - Abreviaturas y caracteres específicos (ej. "I", "F", "'", "u", "us").
- La función `set().union()` combina ambos conjuntos para evitar duplicados y optimizar el filtrado.

Código:

```
# Stopwords y símbolos a eliminar
stop_words = set(stopwords.words('english')).union(set(string.punctuation))
otros_ruidos = set(["'", "'d", "'ll", "'m", "'re", "'s", "'t", "'ve", "--", '000', '1', '2', '3', '4', '5', '6', '8', '10', '15', '30', 'I', 'F', ''])
```

Contexto:

Este paso corresponde a la etapa de limpieza de datos del laboratorio, donde se excluyen términos irrelevantes para construir un vocabulario significativo para la extracción de características (como bolsa de palabras).

Línea 8: Filtrado de formas gramaticales de "hard"

Este código crea un conjunto con las formas léxicas de "hard" (ej. "hard", "harder", "hardest", "die-hard") extraídas de las claves del objeto `formas_hard`, que fue generado previamente (como se vio en el código anterior que analizaba formas gramaticales).

Código:

```
# Formas gramaticales de "hard" que queremos eliminar
formas_hard_palabras = set([w for (w, _) in formas_hard.keys()])
```

Detalles:

- `formas_hard` es un objeto `Counter` que contiene tuplas (palabra, `POS_tag`) como claves (ej. ('hard', 'JJ'), ('harder', 'JJ')).
- La comprensión de lista `[w for (w, _) in formas_hard.keys()]` extrae solo la primera componente de cada tupla (la palabra) e ignora la segunda (el `POS tag`, representado por `_`).
- `set(...)` convierte la lista en un conjunto para eliminar duplicados y facilitar el filtrado.

Contexto:

Excluir las formas gramaticales de "hard" (ej. "hard", "harder") es importante para evitar sesgos en el vocabulario, ya que estas formas están directamente relacionadas con la palabra objetivo y no deberían usarse como características de contexto en la WSD.

Línea 10: Combinación de filtros

Este código combina todos los filtros en un solo conjunto (palabras_omitidas) que incluye stopwords, signos de puntuación, términos de ruido y formas gramaticales de "hard".

Código:

```
# Unir todos los filtros
palabras_omitidas = stop_words.union(otros_ruidos).union(formas_hard_palabras)
```

Detalles:

La función union() asegura que no haya duplicados, creando una lista eficiente de términos a excluir durante el procesamiento del contexto.

Contexto:

Este conjunto se usa para filtrar las palabras del contexto de las instancias, asegurando que el vocabulario final contenga solo palabras relevantes para la desambiguación.

Líneas 12-18: Recolección de tokens válidos

Este código recorre las instancias de "hard" (hard_instances, cargadas previamente con senseval.instances("hard.pos")) y extrae las palabras del contexto que cumplen ciertos criterios, almacenándolas en una lista (tokens_hard).

Código:

```
# Recoger palabras válidas del contexto
tokens_hard = []
for inst in hard_instances:
    for elemento in inst.context:
        if isinstance(elemento, tuple) and len(elemento) == 2:
            word, tag = elemento
            word_lower = word.lower()
            if word_lower not in palabras_omitidas:
                tokens_hard.append(word_lower)
```

Detalles:

- tokens_hard = []: Inicializa una lista vacía para almacenar las palabras válidas.
- for inst in hard_instances: Itera sobre las instancias de "hard" en el corpus Senseval 2.
- for elemento in inst.context: Itera sobre los elementos del contexto de cada instancia, que son tuplas (palabra, POS_tag) o, en casos de error, elementos mal formateados (como "FRASL").
- if isinstance(elemento, tuple) and len(elemento) == 2: Verifica que el elemento sea una tupla con exactamente dos componentes (palabra y POS tag), filtrando instancias mal formateadas (como las detectadas previamente con "FRASL").
- word, tag = elemento: Desempaqueta la tupla en la palabra y su etiqueta gramatical.
- word_lower = word.lower(): Convierte la palabra a minúsculas para normalizarla y facilitar la comparación con palabras_omitidas.
- if word_lower not in palabras_omitidas: Solo incluye la palabra en tokens_hard si no está en el conjunto de palabras a omitir.
- tokens_hard.append(word_lower): Agrega la palabra válida a la lista.

Contexto:

Este paso recopila todas las palabras de los contextos de "hard" que son potencialmente útiles para construir el vocabulario, excluyendo ruido y formas de la palabra objetivo.

Líneas 20-21: Cálculo de las palabras más frecuentes

Este código calcula la distribución de frecuencias de las palabras en tokens_hard y selecciona las 250 palabras más frecuentes para formar el vocabulario.

Código:

```
# Calcular las 250 palabras más frecuentes
fdist_hard = FreqDist(tokens_hard)
vocabulario_hard = list(fdist_hard)[:250]
```

Detalles:

- `fdist_hard = FreqDist(tokens_hard)`: Crea un objeto `FreqDist` que cuenta la frecuencia de cada palabra en la lista `tokens_hard`. Este objeto es similar a un diccionario, donde las claves son las palabras y los valores son sus frecuencias.
- `list(fdist_hard)[:250]`: Convierte las claves de `fdist_hard` (las palabras) en una lista y toma las primeras 250, que corresponden a las más frecuentes (ordenadas automáticamente por `FreqDist` en orden descendente de frecuencia).

Contexto:

Este vocabulario de 250 palabras es el que se utiliza en el laboratorio para construir vectores de características basados en bolsa de palabras, como se mencionó en la etapa de extracción de características.

Líneas 23-24: Impresión del vocabulario

Este código muestra las primeras 10 palabras del vocabulario generado para "hard" para inspeccionar los resultados.

Código:

```
print("Primeras 10 palabras del vocabulario para 'hard':")
print(vocabulario_hard[:10])
```

Salida observada:

```
Primeras 10 palabras del vocabulario para 'hard':
['may', 'lose', 'popular', 'support', 'someone', 'kill', 'defeat', 'clever', 'white', 'house']
```

Parte 6: Función para extraer el vector de características


```
[8]: def vector_caracteristicas_vecinas(instancia, vocabulario):
    contexto = set()
    for elemento in instancia.context:
        if isinstance(elemento, tuple) and len(elemento) == 2:
            word, _ = elemento
            contexto.add(word.lower())

    caracteristicas = {}
    for palabra in vocabulario:
        caracteristicas[f"contains({palabra})"] = (palabra in contexto)
    return caracteristicas

# Ejemplo con la primera instancia de "hard"
ejemplo_instancia = hard_instances[0]
vector = vector_caracteristicas_vecinas(ejemplo_instancia, vocabulario_hard)

# Mostrar solo las primeras 10 características para verificar
print("Vector de características (parcial):")
for i, (k, v) in enumerate(vector.items()):
    if i >= 10:
        break
    print(f"{k}: {v}")

Vector de características (parcial):
contains(may): True
contains(lose): True
contains(popular): True
contains(support): True
contains(someone): True
contains(kill): True
contains(defeat): True
contains(clever): False
contains(white): False
contains(house): False
```

El código define una función que genera un vector de características de bolsa de palabras para una instancia de "hard", indicando la presencia/ausencia de las palabras del vocabulario `vocabulario_hard` en su contexto. La salida muestra que las palabras "may", "lose", "popular", "support", "someone", "kill" y "defeat" están presentes en el contexto de la primera instancia, lo que sugiere un contexto semánticamente relevante para la WSD. Este vector es clave para el clasificador Naive Bayes basado en palabras vecinas.

Líneas 1-9: Definición de la función `vector_caracteristicas_vecinas`

Este código define una función reusable que genera un vector de características para una instancia de Senseval 2, indicando si cada palabra del vocabulario está presente en el contexto de la instancia.

Código:

```
[8]: def vector_caracteristicas_vecinas(instancia, vocabulario):
    contexto = set()
    for elemento in instancia.context:
        if isinstance(elemento, tuple) and len(elemento) == 2:
            word, _ = elemento
            contexto.add(word.lower())

    caracteristicas = {}
    for palabra in vocabulario:
        caracteristicas[f"contains({palabra})"] = (palabra in contexto)
    return caracteristicas
```

Parámetros:

- `instancia`: Un objeto `SensevalInstance` del corpus Senseval 2, que contiene el contexto (lista de tuplas (palabra, POS_tag)), la palabra objetivo, el sentido etiquetado y la posición de la palabra en el contexto.
- `vocabulario`: Lista de palabras (previamente generada, ej. `vocabulario_hard`) que define el conjunto de términos a considerar en el vector de características.

- `contexto = set()`: Inicializa un conjunto vacío para almacenar las palabras del contexto de la instancia en minúsculas, eliminando duplicados automáticamente gracias a la estructura de conjunto (`set`).
- `for elemento in instancia.context`: Itera sobre los elementos del contexto de la instancia.
- `if isinstance(elemento, tuple) and len(elemento) == 2`: Verifica que el elemento sea una tupla con exactamente dos componentes (palabra y POS tag), filtrando elementos mal formateados (como "FRASL", identificado en el laboratorio como un error).
- `word, _ = elemento`: Desempaqueta la tupla en la palabra y su etiqueta POS, ignorando la etiqueta (`_` indica que no se usa).
- `contexto.add(word.lower())`: Convierte la palabra a minúsculas y la agrega al conjunto contexto, asegurando normalización y eliminación de duplicados.
- `caracteristicas = {}`: Inicializa un diccionario vacío para almacenar el vector de características.
- `for palabra in vocabulario`: Itera sobre cada palabra del vocabulario.
- `caracteristicas[f"contains({palabra})"] = (palabra in contexto)`: Crea una entrada en el diccionario con la clave `contains(palabra)` y un valor booleano (`True` si la palabra está en el contexto, `False` si no).
- `return caracteristicas`: Devuelve el diccionario que representa el vector de características.

Contexto:

Esta función implementa el modelo de bolsa de palabras descrito en el laboratorio, donde cada instancia se representa como un vector binario que indica la presencia/ausencia de las palabras del vocabulario en el contexto. Este vector es compatible con los clasificadores Naive Bayes de NLTK.

Líneas 11-12: Generación del vector de características para una instancia

Este código selecciona la primera instancia de "hard" (`hard_instances[0]`, cargada previamente con `senseval.instances('hard.pos')`) y genera su vector de características usando el vocabulario `vocabulario_hard` (las 250 palabras más frecuentes construidas previamente).

Código:

```
# Ejemplo con la primera instancia de "hard"
ejemplo_instancia = hard_instances[0]
vector = vector_caracteristicas_vecinas(ejemplo_instancia, vocabulario_hard)
```

Detalles:

- `hard_instances[0]`: La primera instancia de "hard" en el corpus, que contiene el contexto, el sentido (ej. HARD1, HARD2, o HARD3), y otros atributos.
- `vocabulario_hard`: Lista de las 250 palabras más frecuentes en los contextos de "hard", generada en el código anterior (ej. ['may', 'lose', 'popular', ...]).
- `vector`: Un diccionario con 250 entradas, cada una indicando si una palabra del vocabulario está presente en el contexto de la instancia.

Contexto:

Este paso prueba la función en una sola instancia para verificar su correcto funcionamiento antes de aplicarla a todas las instancias en el entrenamiento del clasificador.

Líneas 14-18: Impresión de las primeras 10 características

Este código muestra las primeras 10 entradas del vector de características para inspeccionar el resultado.

Código:

```
# Mostrar solo las primeras 10 características para verificar
print("Vector de características (parcial):")
for i, (k, v) in enumerate(vector.items()):
    if i >= 10:
        break
    print(f"{k}: {v}")
```

Detalles:

- `vector.items()`: Devuelve las tuplas (clave, valor) del diccionario `vector`.
- `enumerate(vector.items())`: Itera sobre las entradas con un índice para limitar la salida a las primeras 10.
- `if i >= 10: break`: Detiene el bucle después de imprimir 10 características para mantener la salida manejable.
- `print(f"{k}: {v}")`: Imprime cada clave (ej. `contains(may)`) y su valor booleano (`True` o `False`).

Salida observada:

```
Vector de características (parcial):
contains(may): True
contains(lose): True
contains(popular): True
contains(support): True
contains(someone): True
contains(kill): True
contains(defeat): True
contains(clever): False
contains(white): False
contains(house): False
```

Paso 7: Extracción de características basada en colocaciones (n-gramas)

```
def vector_colocaciones(instancia, n=2):
    contexto = list(instancia.context)
    pos_objetivo = instancia.position # posición de la palabra ambigua
    palabras = [w.lower() for (w, t) in contexto if isinstance((w, t), tuple)]

    características = {}

    # Bigramas anteriores
    if pos_objetivo >= n:
        anterior = " ".join(palabras[pos_objetivo - n:pos_objetivo])
        características[f"previous({anterior})"] = True
    else:
        anterior = " ".join(palabras[:pos_objetivo])
        características[f"previous({anterior})"] = True

    # Bigramas posteriores
    if pos_objetivo + n < len(palabras):
        posterior = " ".join(palabras[pos_objetivo + 1:pos_objetivo + 1 + n])
        características[f"next({posterior})"] = True
    else:
        posterior = " ".join(palabras[pos_objetivo + 1:])
        características[f"next({posterior})"] = True

    return características

# Prueba con la primera instancia de "hard"
vector_col = vector_colocaciones(hard_instances[0])

print("Vector de colocaciones:")
for k, v in vector_col.items():
    print(f"{k}: {v}")
```

El código define una función que genera un vector de características basado en bigramas (colocaciones) para la primera instancia de "hard" en el corpus Senseval 2. La salida muestra los bigramas "that 's" (anterior) y "to do" (posterior), que son relevantes para el contexto de la instancia, aunque "that 's" es menos informativo. Este vector es clave para el clasificador Naive Bayes basado en colocaciones, que superó al modelo de bolsa de palabras en el laboratorio. Las mejoras sugeridas podrían aumentar la robustez y especificidad de las características.

Líneas 1-19: Definición de la función `vector_colocaciones`

Este código define una función reusable que genera un vector de características basado en colocaciones (bigramas) para una instancia de Senseval 2, capturando los pares de palabras inmediatamente anteriores y posteriores a la palabra objetivo (en este caso, "hard").

Código:

```
[9]: def vector_colocaciones(instancia, n=2):
    contexto = list(instancia.context)
    pos_objetivo = instancia.position # posición de la palabra ambigua
    palabras = [w.lower() for (w, t) in contexto if isinstance((w, t), tuple)]

    características = {}

    # Bigramas anteriores
    if pos_objetivo >= n:
        anterior = " ".join(palabras[pos_objetivo - n:pos_objetivo])
        características[f"previous({anterior})"] = True
    else:
        anterior = " ".join(palabras[:pos_objetivo])
        características[f"previous({anterior})"] = True

    # Bigramas posteriores
    if pos_objetivo + n < len(palabras):
        posterior = " ".join(palabras[pos_objetivo + 1:pos_objetivo + 1 + n])
        características[f"next({posterior})"] = True
    else:
        posterior = " ".join(palabras[pos_objetivo + 1:])
        características[f"next({posterior})"] = True

    return características
```

Parámetros:

- `instancia`: Un objeto `SensevalInstance` del corpus Senseval 2, que contiene el contexto (lista de tuplas (palabra, POS_tag)), la palabra objetivo, el sentido etiquetado y la posición de la palabra en el contexto.
- `n=2`: El número de palabras a considerar para los bigramas (por defecto, 2, lo que significa bigramas: pares de palabras consecutivas).
- `contexto = list(instancia.context)`: Convierte el atributo `context` de la instancia (que contiene el texto circundante de la palabra objetivo) en una lista de tuplas (palabra, POS_tag) o elementos mal formateados (como "FRASL").
- `pos_objetivo = instancia.position`: Obtiene la posición de la palabra objetivo ("hard") en el contexto, que indica su índice en la lista de palabras.
- `palabras = [w.lower() for (w, t) in contexto if isinstance((w, t), tuple)]`: Crea una lista de palabras en minúsculas, filtrando solo los elementos del contexto que son tuplas válidas (excluyendo errores como "FRASL"). Ignora las etiquetas POS (t) usando una comprensión de lista.

- `caracteristicas = {}`: Inicializa un diccionario vacío para almacenar el vector de características.

Bigramas anteriores:

- `if pos_objetivo >= n`: Verifica si hay al menos `n` (2) palabras antes de la palabra objetivo.
- `anterior = " ".join(palabras[pos_objetivo - n:pos_objetivo])`: Toma las `n` palabras anteriores a la palabra objetivo y las une con un espacio para formar un bigrama (ej. "word1 word2").
- Si no hay suficientes palabras antes (`pos_objetivo < n`), toma todas las palabras disponibles desde el inicio del contexto (`palabras[:pos_objetivo]`).
- `caracteristicas[f"previous({anterior})"] = True`: Agrega una entrada al diccionario indicando que el bigrama anterior está presente.

Bigramas posteriores:

- `if pos_objetivo + n < len(palabras)`: Verifica si hay al menos `n` palabras después de la palabra objetivo.
- `posterior = " ".join(palabras[pos_objetivo + 1:pos_objetivo + 1 + n])`: Toma las `n` palabras posteriores (excluyendo la palabra objetivo) y las une con un espacio.
- Si no hay suficientes palabras después, toma todas las palabras disponibles hasta el final del contexto (`palabras[pos_objetivo + 1:]`).
- `caracteristicas[f"next({posterior})"] = True`: Agrega una entrada indicando que el bigrama posterior está presente.
- `return caracteristicas`: Devuelve el diccionario con dos entradas: una para el bigrama anterior (`previous(...)`) y otra para el bigrama posterior (`next(...)`).

Contexto:

Esta función implementa el enfoque de colocaciones descrito en el laboratorio, donde los bigramas (pares de palabras consecutivas antes y después de la palabra objetivo) se utilizan como características para los clasificadores Naive Bayes. Este enfoque resultó más efectivo que la bolsa de palabras, logrando una exactitud de 0.8818 para "hard" frente a 0.7891.

Líneas 21-22: Generación del vector de colocaciones (continuación)

Este código aplica la función `vector_colocaciones` a la primera instancia de "hard" (`hard_instances[0]`, cargada previamente con `senseval.instances('hard.pos')`) para generar un vector de características que incluye los bigramas anteriores y posteriores a la palabra objetivo.

Código:

```
# Prueba con la primera instancia de "hard"
vector_col = vector_colocaciones(hard_instances[0])
```

Detalles:

- `hard_instances[0]`: La primera instancia de "hard" en el corpus, un objeto `SensevalInstance` que contiene:
- `word`: La palabra objetivo ("hard").
- `context`: Lista de tuplas (palabra, POS_tag) o elementos mal formateados (como "FRASL", filtrados por la función).
- `senses`: Lista de sentidos etiquetados (ej. HARD1, HARD2, HARD3).
- `position`: Índice de la palabra objetivo en el contexto.
- `vector_col`: Un diccionario con dos entradas, una para el bigrama anterior (`previous(...)`) y otra para el bigrama posterior (`next(...)`), cada una con el valor `True` para indicar su presencia.

Contexto:

Este paso prueba la función en una sola instancia para verificar su correcto funcionamiento antes de aplicarla a todas las instancias en el entrenamiento del clasificador Naive Bayes. En el laboratorio, este enfoque de colocaciones logró una exactitud de 0.8818 para "hard", superando al modelo de bolsa de palabras (0.7891).

Líneas 24-26: Impresión del vector de colocaciones

Este código muestra el contenido completo del vector de características generado por `vector_colocaciones`, imprimiendo cada clave (bigrama) y su valor booleano.

Código:

```
print("Vector de colocaciones:")
for k, v in vector_col.items():
    print(f"{k}: {v}")
```

Detalles:

- `vector_col.items()`: Devuelve las tuplas (clave, valor) del diccionario `vector_col`.
- `for k, v in vector_col.items()`: Itera sobre las entradas del diccionario, imprimiendo cada clave (en el formato `previous(...)` o `next(...)`) y su valor (`True`).
- A diferencia del código anterior (que limitaba la salida a 10 características), aquí se imprimen todas las entradas, aunque en este caso solo hay dos (un bigrama anterior y uno posterior).

Salida observada:

```
Vector de colocaciones:
previous(that 's): True
next(to do): True
```

Interpretación:

- `previous(that 's): True`: Indica que las dos palabras inmediatamente anteriores a "hard" en el contexto de la primera instancia forman el bigrama "that 's" (probablemente una contracción de "that is"). Esto sugiere que el contexto podría ser algo como "that is hard ..." o "that's hard ...".
- `next(to do): True`: Indica que las dos palabras inmediatamente posteriores a "hard" forman el bigrama "to do", sugiriendo un contexto como "hard to do ...".
- El vector tiene solo dos características, lo que es esperado, ya que la función genera un bigrama anterior y un bigrama posterior por instancia (asumiendo que hay suficientes palabras en el contexto). Los valores `True` confirman que ambos bigramas están presentes.
- Contexto semántico: El bigrama "to do" sugiere que "hard" podría estar en un contexto relacionado con una tarea o acción difícil (ej. "hard to do something"), lo que probablemente corresponde al sentido HARD2 ("dificultad").

Paso 8: Preparar funciones de extracción de características

```
[14]: def extraer_vecinos(instancia, vocabulario):
    contexto = [w.lower() for (w, t) in instancia.context if isinstance((w, t), tuple)]
    return {f"contains({pal})": (pal in contexto) for pal in vocabulario}

def extraer_colocacion(instancia, n=2):
    contexto = list(instancia.context)
    pos = instancia.position
    contexto_palabras = [w for (w, t) in contexto if isinstance((w, t), tuple)]

    prev = " ".join(contexto_palabras[max(0, pos - n):pos])
    next_ = " ".join(contexto_palabras[pos + 1:pos + 1 + n])

    features = {}
    if prev:
        features[f"previous({prev})"] = True
    if next_:
        features[f"next({next_})"] = True
    return features
```

Este código define dos funciones para generar vectores de características para el laboratorio de WSD:

- **extraer_vecinos:** Crea un vector de bolsa de palabras con 250 características (presencia/ausencia de palabras del vocabulario), similar a la función `vector_caracteristicas_vecinas` pero más concisa.
- **extraer_colocacion:** Crea un vector con hasta dos características (bigramas anterior y posterior), similar a `vector_colocaciones` pero sin normalización a minúsculas.

Ambas funciones son esenciales para los clasificadores Naive Bayes, con las colocaciones ofreciendo mejor rendimiento debido a su especificidad. Las mejoras sugeridas (normalización, POS tags, filtrado de stopwords) podrían aumentar la robustez y efectividad de los vectores, alineándose con las propuestas del laboratorio.

A continuación, se desglosa cada función y su propósito:

1. Función `extraer_vecinos`

Propósito: Genera un vector de características basado en el modelo de bolsa de palabras (Bag of Words), que indica la presencia o ausencia de cada palabra del vocabulario en el contexto de una instancia.

Código:

```
[14]: def extraer_vecinos(instancia, vocabulario):
    contexto = [w.lower() for (w, t) in instancia.context if isinstance((w, t), tuple)]
    return {f"contains({pal})": (pal in contexto) for pal in vocabulario}
```

Parámetros:

- **instancia:** Un objeto `SensevalInstance` del corpus `Senseval 2`, que contiene:
- **word:** La palabra objetivo (ej. "hard" o "serve").
- **context:** Lista de tuplas (palabra, POS_tag) que representan el contexto, o elementos mal formateados (como "FRASL").
- **senses:** Lista de sentidos etiquetados (ej. HARD1, SERVE10).
- **position:** Índice de la palabra objetivo en el contexto.
- **vocabulario:** Lista de palabras (ej. `vocabulario_hard`, las 250 palabras más frecuentes generadas previamente) que define los términos a considerar en el vector.
- **contexto = [w.lower() for (w, t) in instancia.context if isinstance((w, t), tuple)]:** Crea una lista de palabras en minúsculas extraídas del contexto, filtrando elementos que no sean tuplas.

válidas (palabra, POS_tag). Esto excluye errores como "FRASL", identificados previamente en el laboratorio.

- `return {f"contains({pal})": (pal in contexto) for pal in vocabulario}`: Utiliza una comprensión de diccionario para generar un diccionario donde:
 - Las claves son cadenas en el formato `contains(palabra)` (ej. `contains(may)`).
 - Los valores son booleanos (True si la palabra está en el contexto, False si no).
- Salida: Un diccionario con una entrada por cada palabra en el vocabulario (ej. 250 entradas si `vocabulario_hard` tiene 250 palabras), representando el vector de características de bolsa de palabras.

Contexto:

Esta función implementa el enfoque de palabras vecinas descrito en el laboratorio, donde se genera un vector binario que captura el contexto léxico de la palabra objetivo. En el laboratorio, este modelo logró una exactitud de 0.7891 para "hard" y 0.6713 para "serve", inferior al enfoque de colocaciones.

Diferencias con el código previo:

Comparada con la función `vector_caracteristicas_vecinas` analizada anteriormente, esta versión es más concisa al usar una comprensión de diccionario en lugar de un bucle explícito, pero tiene la misma funcionalidad. No utiliza un conjunto (set) para el contexto, lo que permite contar palabras repetidas (aunque el operador `in` en la comprensión ignora duplicados al devolver un booleano).

2. Función `extraer_colocacion`

Este código genera un vector de características basado en colocaciones, capturando los bigramas (o secuencias de n palabras) inmediatamente anteriores y posteriores a la palabra objetivo en el contexto.

Código:

```
def extraer_colocacion(instancia, n=2):
    contexto = list(instancia.context)
    pos = instancia.position
    contexto_palabras = [w for (w, t) in contexto if isinstance((w, t), tuple)]

    prev = " ".join(contexto_palabras[max(0, pos - n):pos])
    next_ = " ".join(contexto_palabras[pos + 1:pos + 1 + n])

    features = {}
    if prev:
        features[f"previous({prev})"] = True
    if next_:
        features[f"next({next_})"] = True
    return features
```

Parámetros:

- `instancia`: Un objeto `SensevalInstance`, similar al de `extraer_vecinos`.
- `n=2`: El número de palabras a considerar para los bigramas (por defecto, 2, lo que genera bigramas).
- `contexto = list(instancia.context)`: Convierte el atributo `context` en una lista de tuplas (palabra, POS_tag) o elementos mal formateados.
- `pos = instancia.position`: Obtiene el índice de la palabra objetivo en el contexto.

- `contexto_palabras = [w for (w, t) in contexto if isinstance((w, t), tuple)]`: Crea una lista de palabras (sin POS tags) filtrando elementos no válidos, similar a `extraer_vecinos`, pero sin convertir a minúsculas (un detalle que podría mejorarse para consistencia).
- `prev = " ".join(contexto_palabras[max(0, pos - n):pos])`: Genera el bigrama anterior tomando las `n` palabras antes de la palabra objetivo (o todas las disponibles si hay menos de `n`). La función `max(0, pos - n)` asegura que el índice no sea negativo.
- `next_ = " ".join(contexto_palabras[pos + 1:pos + 1 + n])`: Genera el bigrama posterior tomando las `n` palabras después de la palabra objetivo (o todas las disponibles si no hay suficientes).
- `features = {}`: Inicializa un diccionario vacío para las características.
- `if prev: features[f"previous({prev})"] = True`: Agrega el bigrama anterior al diccionario si existe (si `prev` no es una cadena vacía).
- `if next_: features[f"next({next_})"] = True`: Agrega el bigrama posterior si existe.
- `return features`: Devuelve un diccionario con hasta dos entradas: una para el bigrama anterior (`previous(...)`) y una para el bigrama posterior (`next(...)`), cada una con valor `True`.

Contexto:

Esta función implementa el enfoque de colocaciones descrito en el laboratorio, que captura relaciones semánticas más específicas al considerar el orden y la proximidad de las palabras. En el laboratorio, este enfoque logró una exactitud de 0.8818 para "hard" y 0.7282 para "serve", superando a las palabras vecinas.

Diferencias con el código previo:

Comparada con la función `vector_colocaciones` analizada anteriormente, esta versión es más concisa al usar comprensiones de lista y manejar los bigramas en una sola línea. Sin embargo, no convierte las palabras a minúsculas (lo que podría introducir inconsistencias) y usa nombres de variables diferentes (`prev`, `next_` en lugar de `anterior`, `posterior`). La lógica para manejar contextos cortos es ligeramente diferente, pero el resultado es equivalente.

Paso 9: Entrenamiento y evaluación de clasificadores para "hard"

```
[15]: from nltk.classify import NaiveBayesClassifier
      from nltk.classify.util import accuracy
      from nltk import ConfusionMatrix
      import random

      # Filtramos instancias mal formateadas
      hard_filtrado = [i for i in hard_instances if isinstance(i.context, list) and all(isinstance(x, tuple) and len(x) == 2 for x in i.context)]

      # Barajamos aleatoriamente y separamos 80/20
      random.seed(42)
      random.shuffle(hard_filtrado)
      cutoff = int(0.8 * len(hard_filtrado))
      train_hard = hard_filtrado[:cutoff]
      test_hard = hard_filtrado[cutoff:]

      # Entrenamiento con palabras vecinas
      train_vecinos = [(extraer_vecinos(i, vocabulario_hard), i.senses[0]) for i in train_hard]
      test_vecinos = [(extraer_vecinos(i, vocabulario_hard), i.senses[0]) for i in test_hard]

      clf_vecinos = NaiveBayesClassifier.train(train_vecinos)
      print("Exactitud (vecinos) - 'hard':", accuracy(clf_vecinos, test_vecinos))

      # Entrenamiento con colocaciones
      train_coloc = [(extraer_colocacion(i), i.senses[0]) for i in train_hard]
      test_coloc = [(extraer_colocacion(i), i.senses[0]) for i in test_hard]

      clf_coloc = NaiveBayesClassifier.train(train_coloc)
      print("Exactitud (colocación) - 'hard':", accuracy(clf_coloc, test_coloc))

      # Matriz de confusión para colocación
      truth = [label for (feats, label) in test_coloc]
      preds = [clf_coloc.classify(feats) for (feats, label) in test_coloc]
      print("\nMatriz de confusión (colocación) - 'hard':")
      print(ConfusionMatrix(truth, preds))

      Exactitud (vecinos) - 'hard': 0.7891877636152954
      Exactitud (colocación) - 'hard': 0.8818076477404483

      Matriz de confusión (colocación) - 'hard':
      | H  W  H |
      | A  A  A |
      | R  R  R |
      | D  D  D |
      | 1  2  3 |

      -----
      HARD1 |<673> 4 9 |
      HARD2 | 43 <58> 3 |
      HARD3 | 39  4 <38> |
      -----
      (row = reference; col = test)
```

Este código filtra instancias mal formateadas de "hard", divide los datos en entrenamiento (80%) y prueba (20%), entrena dos clasificadores Naive Bayes (uno con palabras vecinas, otro con colocaciones), y evalúa su rendimiento. La salida confirma las exactitudes del laboratorio (0.7891 para palabras vecinas, 0.8818 para colocaciones) y muestra una matriz de confusión que destaca los errores en sentidos menos frecuentes (HARD2, HARD3) debido al desbalance. Este código es la culminación de las etapas de limpieza, extracción de características y clasificación del laboratorio, validando la superioridad de las colocaciones.

A continuación, se desglosa cada parte del código:

Líneas 1-4: Importaciones

En este caso el propósito de este código se basa en:

- **NaiveBayesClassifier**: Importa la clase para entrenar un clasificador Naive Bayes, que asume independencia condicional entre las características y es adecuado para tareas de clasificación de texto como WSD.
- **accuracy**: Importa una función de NLTK para calcular la exactitud del clasificador comparando las etiquetas predichas con las reales.
- **ConfusionMatrix**: Importa la clase para generar matrices de confusión, que muestran el rendimiento del clasificador por clase (sentido).
- **random**: Importa el módulo estándar de Python para barajar los datos aleatoriamente y garantizar reproducibilidad.
- **Contexto**: Estas herramientas son esenciales para la etapa de entrenamiento y evaluación del laboratorio, permitiendo implementar y analizar los clasificadores descritos.

Código:

```
[15]: from nltk.classify import NaiveBayesClassifier
      from nltk.classify.util import accuracy
      from nltk import ConfusionMatrix
      import random
```

Líneas 6-7: Filtrado de instancias mal formateadas

Este código filtra las instancias de "hard" (`hard_instances`, cargadas previamente con `senseval.instances('hard.pos')`) para excluir aquellas con contextos mal formateados, como las identificadas previamente con "FRASL".

Código:

```
# Filtramos instancias mal formateadas
hard_filtrado = [i for i in hard_instances if isinstance(i.context, list) and all(isinstance(x, tuple) and len(x) == 2 for x in i.context)]
```

Detalles:

- `isinstance(i.context, list)`: Verifica que el atributo `context` de la instancia sea una lista.
- `all(isinstance(x, tuple) and len(x) == 2 for x in i.context)`: Asegura que todos los elementos del contexto sean tuplas de dos elementos (palabra, POS_tag).
- La comprensión de lista crea una nueva lista (`hard_filtrado`) con solo las instancias válidas.

Contexto:

Este paso corresponde a la etapa de limpieza de datos del laboratorio, eliminando las 19 instancias mal formateadas de "hard" (identificadas previamente). Esto asegura que los datos usados para el entrenamiento sean válidos, reduciendo el ruido.

Líneas 9-12: División de datos en entrenamiento y prueba

Propósito: Baraja aleatoriamente las instancias filtradas y las divide en un conjunto de entrenamiento (80%) y un conjunto de prueba (20%).

Código:

```
# Barajamos aleatoriamente y separamos 80/20
random.seed(42)
random.shuffle(hard_filtrado)
cutoff = int(0.8 * len(hard_filtrado))
train_hard = hard_filtrado[:cutoff]
test_hard = hard_filtrado[cutoff:]
```

Detalles:

- `random.seed(42)`: Establece una semilla para garantizar que el barajado sea reproducible.
- `random.shuffle(hard_filtrado)`: Reordena aleatoriamente las instancias en `hard_filtrado`.
- `cutoff = int(0.8 * len(hard_filtrado))`: Calcula el índice de corte para dividir el 80% (entrenamiento) y el 20% (prueba). Dado que "hard" tiene 4333 instancias totales (3455 + 502 + 376, según el conteo previo) y 19 mal formateadas, `hard_filtrado` tiene 4314 instancias, por lo que `cutoff ≈ 3451`.
- `train_hard = hard_filtrado[:cutoff]`: Toma las primeras 3451 instancias para entrenamiento.
- `test_hard = hard_filtrado[cutoff:]`: Toma las 863 instancias restantes para prueba.

Contexto:

Esta división 80/20 es estándar en aprendizaje automático para entrenar y evaluar modelos, asegurando que el clasificador se evalúe en datos no vistos. La semilla garantiza resultados consistentes con los reportados en el laboratorio (exactitudes de 0.7891 y 0.8818).

Líneas 14-17: Preparación y entrenamiento con palabras vecinas

Este código prepara los datos de entrenamiento y prueba usando el enfoque de palabras vecinas (bolsa de palabras), entrena un clasificador Naive Bayes, y calcula su exactitud.

Código:

```
# Entrenamiento con palabras vecinas
train_vecinos = [(extraer_vecinos(i, vocabulario_hard), i.senses[0]) for i in train_hard]
test_vecinos = [(extraer_vecinos(i, vocabulario_hard), i.senses[0]) for i in test_hard]

clf_vecinos = NaiveBayesClassifier.train(train_vecinos)
print("Exactitud (vecinos) - 'hard':", accuracy(clf_vecinos, test_vecinos))
```

Detalles:

- `train_vecinos = [(extraer_vecinos(i, vocabulario_hard), i.senses[0]) for i in train_hard]`: Crea una lista de tuplas (características, etiqueta) para el conjunto de entrenamiento, donde:

- `extraer_vecinos(i, vocabulario_hard)`: Genera un vector de características de bolsa de palabras (como se definió previamente), con 250 entradas indicando la presencia/ausencia de palabras del vocabulario.
- `i.senses[0]`: Extrae el primer (y único) sentido de la instancia (ej. HARD1, HARD2, HARD3).
- `test_vecinos`: Similar, pero para el conjunto de prueba.
- `clf_vecinos = NaiveBayesClassifier.train(train_vecinos)`: Entrena un clasificador Naive Bayes usando los datos de entrenamiento, modelando las probabilidades condicionales de las características dadas las clases (sentidos).
- `accuracy(clf_vecinos, test_vecinos)`: Calcula la exactitud como la proporción de instancias en el conjunto de prueba clasificadas correctamente.

Salida observada:

```
Exactitud (vecinos) - 'hard': 0.7891077636152954
```

Esto indica que el clasificador basado en palabras vecinas clasificó correctamente el 78.91% de las instancias de prueba (aproximadamente 681 de 863 instancias), consistente con la exactitud de 0.7891 reportada en el laboratorio.

Contexto:

Este resultado refleja el rendimiento del modelo de bolsa de palabras, que es menos efectivo que las colocaciones debido a su menor especificidad para capturar relaciones semánticas.

Líneas 19-22: Preparación y entrenamiento con colocaciones

Este código prepara los datos de entrenamiento y prueba usando el enfoque de colocaciones (bigramas), entrena un clasificador Naive Bayes, y calcula su exactitud.

Código:

```
# Entrenamiento con colocaciones
train_coloc = [(extraer_colocacion(i), i.senses[0]) for i in train_hard]
test_coloc = [(extraer_colocacion(i), i.senses[0]) for i in test_hard]

clf_coloc = NaiveBayesClassifier.train(train_coloc)
print("Exactitud (colocación) - 'hard':", accuracy(clf_coloc, test_coloc))
```

Detalles:

- `train_coloc = [(extraer_colocacion(i), i.senses[0]) for i in train_hard]`: Crea una lista de tuplas (características, etiqueta) para el entrenamiento, donde:
- `extraer_colocacion(i)`: Genera un vector de características con hasta dos entradas (bigrama anterior y posterior), como se definió previamente.
- `i.senses[0]`: Extrae el sentido de la instancia.
- `test_coloc`: Similar, pero para el conjunto de prueba.
- `clf_coloc = NaiveBayesClassifier.train(train_coloc)`: Entrena un clasificador Naive Bayes con los datos de colocaciones.
- `accuracy(clf_coloc, test_coloc)`: Calcula la exactitud en el conjunto de prueba.

Salida observada:

```
Exactitud (colocación) - 'hard': 0.8818076477404403
```

Esto indica que el clasificador basado en colocaciones clasificó correctamente el 88.18% de las instancias de prueba (aproximadamente 761 de 863 instancias), consistente con la exactitud de 0.8818 reportada en el laboratorio.

Contexto:

Este resultado confirma que las colocaciones son más efectivas que las palabras vecinas, ya que capturan relaciones semánticas más específicas (como "to do" para HARD2), como se discutió en el laboratorio.

Líneas 24-27: Matriz de confusión para colocaciones

Este código genera y muestra una matriz de confusión para evaluar el rendimiento del clasificador basado en colocaciones, mostrando cómo se distribuyen las predicciones frente a las etiquetas reales.

Código:

```
# Matriz de confusión para colocación
truth = [label for (feats, label) in test_coloc]
preds = [clf_coloc.classify(feats) for (feats, label) in test_coloc]
print("\nMatriz de confusión (colocación) - 'hard':")
print(ConfusionMatrix(truth, preds))
```

Detalles:

- `truth = [label for (feats, label) in test_coloc]`: Extrae las etiquetas reales (sentidos) del conjunto de prueba.
- `preds = [clf_coloc.classify(feats) for (feats, label) in test_coloc]`: Genera las predicciones del clasificador para cada instancia de prueba.
- `ConfusionMatrix(truth, preds)`: Crea una matriz de confusión donde las filas representan las etiquetas reales (row = reference) y las columnas las etiquetas predichas (col = test).

Salida observada:

```
Matriz de confusión (colocación) - 'hard':
      |  H   H   H |
      |  A   A   A |
      |  R   R   R |
      |  D   D   D |
      |  1   2   3 |
-----+-----+
HARD1 |<673> 4   9 |
HARD2 | 43 <58> 3   |
HARD3 | 39  4 <30> |
-----+-----+
(row = reference; col = test)
```

Interpretación:

- Filas: Representan los sentidos reales (HARD1, HARD2, HARD3).
- Columnas: Representan los sentidos predichos.
- Diagonal (en < >): Instancias clasificadas correctamente (673 para HARD1, 58 para HARD2, 30 para HARD3).

- Fuera de la diagonal: Errores de clasificación (ej. 43 instancias de HARD2 predichas como HARD1).
- Distribución en el conjunto de prueba:
 - HARD1: $673 + 4 + 9 = 686$ instancias.
 - HARD2: $43 + 58 + 3 = 104$ instancias.
 - HARD3: $39 + 4 + 30 = 73$ instancias.
 - Total: $686 + 104 + 73 = 863$ instancias, consistente con el 20% de 4314 instancias filtradas.
- Exactitud: $(673 + 58 + 30) / 863 \approx 0.8818$, confirmando la exactitud reportada.

Errores principales:

- HARD2 → HARD1: 43 errores, probablemente debido al desbalance (3455 instancias de HARD1 vs. 502 de HARD2 en el corpus total).
- HARD3 → HARD1: 39 errores, también influido por el desbalance (376 instancias de HARD3).
- HARD1 tiene alta precisión ($673/686 \approx 98\%$), pero HARD2 ($58/104 \approx 56\%$) y HARD3 ($30/73 \approx 41\%$) tienen menor rendimiento, reflejando la dificultad de clasificar sentidos menos frecuentes, como se mencionó en el laboratorio.

Contexto:

La matriz de confusión proporciona una visión detallada de los errores, confirmando que el desbalance de clases y la similitud contextual entre sentidos (ej. HARD2 y HARD3) son desafíos, como se discutió en el laboratorio.

Paso 10: Entrenamiento y evaluación de clasificadores para "serve"

```
[14]: from sklearn.classify import NaiveBayesClassifier
from sklearn.classify.utils import accuracy
from sklearn.metrics import ConfusionMatrix
import random

# Filtramos las instancias bien formateadas para "serve"
serve_clean = [i for i in serve_instancias if all(isinstance(t, tuple) and len(t) == 2 for t in i.context)]

# Construcción del vocabulario para "serve"
tokens_serve = []
for inst in serve_clean:
    for word, tag in inst.context:
        word_lower = word.lower()
        if word_lower not in palabras_entidad:
            tokens_serve.append(word_lower)

fdist_serve = FreqDist(tokens_serve)
vocabulario_serve = list(fdist_serve.coords)

# Función: características vecinas para "serve"
def vecinos_features_serve(instancia):
    contexto = w_lower() for (w, _) in instancia.context if isinstance(_, str)
    features = []
    for palabra in vocabulario_serve:
        features.append('contains({palabra})') = palabra in contexto
    return features

# Función: colocación para "serve"
def colocacion_features_serve(instancia):
    contexto = list(instancia.context)
    posicion = instancia.posicion
    prev_bigram = ' ', join(w_lower() for (w, _) in contexto[:posicion-1])
    next_bigram = ' ', join(w_lower() for (w, _) in contexto[posicion+1:])
    return {
        f'previous({prev_bigram})': True,
        f'next({next_bigram})': True
    }

# Dataset con características vecinas
features_vecinas_serve = [vecinos_features_serve(i), i.sense() for i in serve_clean]
random.shuffle(features_vecinas_serve)
cut = int(len(features_vecinas_serve) * 0.8)
train_set_v, test_set_v = features_vecinas_serve[:cut], features_vecinas_serve[cut:]

# Dataset con colocaciones
features_colocacion_serve = [colocacion_features_serve(i), i.sense() for i in serve_clean]
random.shuffle(features_colocacion_serve)
cut = int(len(features_colocacion_serve) * 0.8)
train_set_c, test_set_c = features_colocacion_serve[:cut], features_colocacion_serve[cut:]

# Entrenamiento
clf_vecinas_serve = NaiveBayesClassifier.train(train_set_v)
clf_colocacion_serve = NaiveBayesClassifier.train(train_set_c)

# Evaluación
acc_vecinas_serve = accuracy(clf_vecinas_serve, test_set_v)
acc_colocacion_serve = accuracy(clf_colocacion_serve, test_set_c)

print(f'Exactitud (vecinas) - "serve": {acc_vecinas_serve}')
print(f'Exactitud (colocacion) - "serve": {acc_colocacion_serve}')

# Matriz de confusión para colocación
gold = [label for c, label in test_set_c]
pred = [clf_colocacion_serve.classify(features) for (features, _) in test_set_c]
print('Matriz de confusión (colocacion) - "serve":')
print(ConfusionMatrix(gold, pred))

Exactitud (vecinas) - "serve": 0.6713124214809884
Exactitud (colocacion) - "serve": 0.7282229561256795

Matriz de confusión (colocacion) - "serve":
      |  S  |  E  |  I  |  A  |
      |----|----|----|----|
      | S | 0 | 0 | 0 |
      | E | 0 | 0 | 0 |
      | I | 0 | 0 | 0 |
      | A | 0 | 0 | 0 |
      |----|----|----|----|
SERV18 | 187 | 17 | 7 | 4 |
SERV12 | 154 | 19 | 3 | 1 |
SERV22 | 40 | 21 | 12 | 1 |
SERV5  | 50 | 3 | 6 | 25 |
      |----|----|----|----|
(row = reference; col = test)
```

Este código filtra instancias de "serve", construye un vocabulario, genera vectores de características (palabras vecinas y colocaciones), entrena clasificadores Naive Bayes, y evalúa su rendimiento. La salida confirma las exactitudes del laboratorio (0.6713 para palabras vecinas, 0.7282 para colocaciones) y muestra una matriz de confusión que destaca los desafíos del desbalance y la clasificación de SERVE6. Las mejoras sugeridas podrían optimizar el vocabulario y la robustez del modelo. A continuación, se desglosa cada parte del código:

Líneas 1-4: Importaciones

El propósito de este código se basa en:

- NaiveBayesClassifier: Clase para entrenar un clasificador Naive Bayes, adecuado para tareas de clasificación de texto como WSD.
- accuracy: Función para calcular la exactitud del clasificador.
- ConfusionMatrix: Clase para generar matrices de confusión, que muestran el rendimiento por clase (sentido).
- random: Módulo para barajar datos aleatoriamente.
- Contexto: Estas herramientas son esenciales para la etapa de entrenamiento y evaluación del laboratorio, similares a las usadas para "hard".

Código:

```
[16]: from nltk.classify import NaiveBayesClassifier
      from nltk.classify.util import accuracy
      from nltk.metrics import ConfusionMatrix
      import random
```

Líneas 6-7: Filtrado de instancias mal formateadas

Este código filtra las instancias de "serve" (serve_instances, cargadas con senseval.instances('serve.pos')) para excluir aquellas con contextos mal formateados (ej. "FRASL", identificado previamente).

Código:

```
# Filtramos las instancias bien formateadas para 'serve'
serve_clean = [i for i in serve_instances if all(isinstance(t, tuple) and len(t) == 2 for t in i.context)]
```

Detalles:

- all(isinstance(t, tuple) and len(t) == 2 for t in i.context): Asegura que todos los elementos del contexto sean tuplas (palabra, POS_tag) de longitud 2.
- Crea una lista serve_clean con instancias válidas.
- Según el análisis previo, "serve" tiene 4378 instancias, de las cuales 76 están mal formateadas, por lo que serve_clean tiene aproximadamente 4302 instancias.

Contexto:

Este paso corresponde a la limpieza de datos del laboratorio, asegurando que solo se usen instancias válidas para evitar ruido en el entrenamiento.

Líneas 9-14: Construcción del vocabulario para "serve"

Este código genera un vocabulario de las 250 palabras más frecuentes en los contextos de "serve", excluyendo palabras no deseadas (stopwords, puntuación, etc.).

Código:

```
# Construcción del vocabulario para "serve"
tokens_serve = []
for inst in serve_clean:
    for word, tag in inst.context:
        word_lower = word.lower()
        if word_lower not in palabras_omitidas:
            tokens_serve.append(word_lower)

fdist_serve = FreqDist(tokens_serve)
vocabulario_serve = list(fdist_serve)[:250]
```

Detalles:

- `tokens_serve = []`: Inicializa una lista para almacenar palabras válidas.
- `for inst in serve_clean`: Itera sobre las instancias filtradas.
- `for word, tag in inst.context`: Itera sobre las tuplas (palabra, POS_tag) del contexto.
- `word_lower = word.lower()`: Normaliza las palabras a minúsculas.
- `if word_lower not in palabras_omitidas`: Filtra palabras en el conjunto `palabras_omitidas` (definido previamente para "hard", incluye stopwords, puntuación, formas de "hard", y ruido como "s", "000").
- `tokens_serve.append(word_lower)`: Agrega palabras válidas a la lista.
- `fdist_serve = FreqDist(tokens_serve)`: Calcula la distribución de frecuencias usando `FreqDist`.
- `vocabulario_serve = list(fdist_serve)[:250]`: Selecciona las 250 palabras más frecuentes.

Contexto:

Este paso es análogo a la construcción de `vocabulario_hard`, pero para "serve". El uso de `palabras_omitidas` (definido para "hard") podría incluir formas irrelevantes (ej. "harder"), lo que sugiere una posible mejora. El vocabulario es usado para el modelo de palabras vecinas.

Líneas 16-22: Función `vecinas_features_serve`

Este código genera un vector de características de bolsa de palabras para una instancia de "serve", indicando la presencia/ausencia de palabras del vocabulario.

Código:


```
# Función: características vecinas para "serve"
def vecinas_features_serve(instance):
    contexto = [w.lower() for (w, _) in instance.context if isinstance(_, str)]
    features = {}
    for palabra in vocabulario_serve:
        features[f'contains({palabra})'] = palabra in contexto
    return features
```

Detalles:

- contexto = [w.lower() for (w, _) in instance.context if isinstance(_, str)]: Crea una lista de palabras en minúsculas, filtrando tuplas donde la segunda componente (POS tag) es una cadena. La condición isinstance(_, str) es redundante, ya que el filtrado en serve_clean asegura que todos los elementos son tuplas válidas.
- features = {}: Inicializa un diccionario para las características.
- for palabra in vocabulario_serve: Itera sobre el vocabulario.
- features[f'contains({palabra})'] = palabra in contexto: Crea una entrada con clave contains(palabra) y valor booleano.
- Devuelve un diccionario con 250 entradas.

Contexto:

Similar a extraer_vecinos para "hard", pero específica para "serve". La redundancia en la verificación de formato podría simplificarse.

Líneas 24-31: Función colocacion_features_serve

Este código genera un vector de características basado en colocaciones (bigramas anterior y posterior) para una instancia de "serve".

Código:

```
# Función: colocaciones para "serve"
def colocacion_features_serve(instance):
    contexto = list(instance.context)
    posicion = instance.position
    prev_bigram = ' '.join(w.lower() for (w, _) in contexto[max(0, posicion-2):posicion])
    next_bigram = ' '.join(w.lower() for (w, _) in contexto[posicion+1:posicion+3])
    return {
        f'previous({prev_bigram})': True,
        f'next({next_bigram})': True
    }
```

Detalles:

- contexto = list(instance.context): Convierte el contexto en una lista.
- posicion = instance.position: Obtiene el índice de la palabra objetivo.
- prev_bigram = ' '.join(w.lower() for (w, _) in contexto[max(0, posicion-2):posicion]): Genera el bigrama anterior (dos palabras antes de "serve").
- next_bigram = ' '.join(w.lower() for (w, _) in contexto[posicion+1:posicion+3]): Genera el bigrama posterior (dos palabras después).

- Devuelve un diccionario con dos entradas, aunque no verifica si los bigramas están vacíos (a diferencia de `extraer_colocacion`).

Contexto:

Similar a `extraer_colocacion` para "hard", pero con una implementación más directa que no maneja explícitamente bigramas vacíos.

Líneas 33-38: Preparación de datos para palabras vecinas

Este código prepara los datos de entrenamiento (80%) y prueba (20%) para el modelo de palabras vecinas.

Código:

```
# Dataset con características vecinas
features_vecinas_serve = [(vecinas_features_serve(i), i.senses[0]) for i in serve_clean]
random.shuffle(features_vecinas_serve)
cut = int(len(features_vecinas_serve) * 0.8)
train_set_v, test_set_v = features_vecinas_serve[:cut], features_vecinas_serve[cut:]
```

Detalles:

- `features_vecinas_serve`: Lista de tuplas (características, etiqueta) para todas las instancias.
- `random.shuffle`: Baraja los datos (sin semilla, lo que podría afectar la reproducibilidad).
- `cut = int(len(features_vecinas_serve) * 0.8)`: Calcula el índice de corte (~3442 instancias para entrenamiento, ~860 para prueba, dado que `serve_clean` tiene 4302 instancias).

Contexto:

Similar a la preparación para "hard", pero sin semilla explícita.

Líneas 40-45: Preparación de datos para colocaciones

Este código prepara los datos para el modelo de colocaciones, similar al paso anterior.

Código:

```
# Dataset con colocaciones
features_colocacion_serve = [(colocacion_features_serve(i), i.senses[0]) for i in serve_clean]
random.shuffle(features_colocacion_serve)
cut = int(len(features_colocacion_serve) * 0.8)
train_set_c, test_set_c = features_colocacion_serve[:cut], features_colocacion_serve[cut:]
```

Detalles:

Igual que para palabras vecinas, pero usando `colocacion_features_serve`.

Líneas 47-48: Entrenamiento

Este código entrena dos clasificadores Naive Bayes, uno para palabras vecinas y otro para colocaciones.

Código:

```
# Entrenamiento
clf_vecinas_serve = NaiveBayesClassifier.train(train_set_v)
clf_colocacion_serve = NaiveBayesClassifier.train(train_set_c)
```

Contexto:

Similar al entrenamiento para "hard", modelando probabilidades condicionales para los sentidos de "serve" (SERVE10, SERVE12, SERVE2, SERVE6).

Líneas 50-53: Evaluación

Este código calcula y muestra la exactitud de ambos clasificadores en el conjunto de prueba.

Código:

```
# Evaluación
acc_vecinas_serve = accuracy(clf_vecinas_serve, test_set_v)
acc_colocacion_serve = accuracy(clf_colocacion_serve, test_set_c)

print(f"Exactitud (vecinos) - 'serve': {acc_vecinas_serve}")
print(f"Exactitud (colocación) - 'serve': {acc_colocacion_serve}")
```

Salida observada:

```
Exactitud (vecinos) - 'serve': 0.6713124274099884
Exactitud (colocación) - 'serve': 0.7282229965156795
```

Esto confirma los resultados del laboratorio, donde las colocaciones superan a las palabras vecinas, aunque ambas tienen menor rendimiento para "serve" que para "hard" (0.7891 y 0.8818), probablemente debido a la mayor cantidad de sentidos (4 vs. 3) y el desbalance.

Líneas 55-58: Matriz de confusión para colocaciones

Este código genera y muestra una matriz de confusión para el clasificador de colocaciones, comparando etiquetas reales con predichas.

Código:

```
# Matriz de confusión para colocación
gold = [label for (_, label) in test_set_c]
pred = [clf_colocacion_serve.classify(features) for (features, _) in test_set_c]
print("\nMatriz de confusión (colocación) - 'serve':")
print(ConfusionMatrix(gold, pred))
```

Salida observada:

```
Matriz de confusión (colocación) - 'serve':
      |  S   S   |
      |  E   E   S   S   |
      |  R   R   E   E   |
      |  V   V   R   R   |
      |  E   E   V   V   |
      |  1   1   E   E   |
      |  0   2   2   6   |
-----+-----
SERVE10 | <307> 17   7   4   |
SERVE12 | 57<173> 16   2   |
SERVE2  | 40  21<121> 1   |
SERVE6  | 60   3   6 <26> |
-----+-----
(row = reference; col = test)
```

Interpretación:

- Filas: Sentidos reales (SERVE10, SERVE12, SERVE2, SERVE6).
- Columnas: Sentidos predichos.
- Diagonal: Instancias correctas (307 para SERVE10, 173 para SERVE12, 121 para SERVE2, 26 para SERVE6).
- Distribución:
 - SERVE10: $307 + 17 + 7 + 4 = 335$ instancias.
 - SERVE12: $57 + 173 + 16 + 2 = 248$ instancias.
 - SERVE2: $40 + 21 + 121 + 1 = 183$ instancias.
 - SERVE6: $60 + 3 + 6 + 26 = 95$ instancias.
 - Total: $335 + 248 + 183 + 95 = 861$ instancias (ligera variación respecto a 860 debido a redondeo en el corte).
- Exactitud: $(307 + 173 + 121 + 26) / 861 \approx 0.7282$, consistente con la salida.
- Errores principales:
 - SERVE10 $\rightarrow$ SERVE12: 17 errores.
 - SERVE12 $\rightarrow$ SERVE10: 57 errores, el mayor error, probablemente por similitud contextual.
 - SERVE2 $\rightarrow$ SERVE10: 40 errores.
 - SERVE6 $\rightarrow$ SERVE10: 60 errores, el más significativo, reflejando la dificultad de clasificar el sentido menos frecuente (439 instancias en el corpus total).

Contexto:

La matriz muestra un desbalance (1814 instancias de SERVE10 vs. 439 de SERVE6 en el corpus), lo que explica el sesgo hacia SERVE10 y la baja precisión para SERVE6 ($26/95 \approx 27\%$).

Paso 11: Instancias mal clasificadas para hard con colocaciones

```
[19]: # Filtramos las instancias bien formateadas (sin 'FRASL' en el contexto)
def es_valida(inst):
    return all(isinstance(elem, tuple) and len(elem) == 2 for elem in inst.context)

hard_clean = [inst for inst in hard_instances if es_valida(inst)]
```

Este código define una función `es_valida` que verifica si los contextos de las instancias de "hard" están bien formateados (tuplas de dos elementos) y filtra las instancias válidas en `hard_clean`, eliminando las 19 instancias con errores como "FRASL". Este paso es crucial para la limpieza de datos en el laboratorio de WSD, asegurando que las etapas de extracción de características y entrenamiento usen datos válidos. La implementación es modular y reutilizable, pero podría beneficiarse de validaciones adicionales y reportes de filtrado. A continuación, se desglosa cada parte del código:

Líneas 1-3: Definición de la función `es_valida`

Este código define una función que verifica si una instancia de Senseval 2 tiene un contexto bien formateado, es decir, si todos los elementos del contexto son tuplas de dos elementos (palabra, POS_tag).

Código:

```
[19]: # Filtramos las instancias bien formateadas (sin 'FRASL' en el contexto)
def es_valida(inst):
    return all(isinstance(elem, tuple) and len(elem) == 2 for elem in inst.context)
```

Parámetro:

- `inst`: Un objeto `SensevalInstance` que contiene:
 - `word`: La palabra objetivo ("hard").
 - `context`: Lista de elementos que deberían ser tuplas (palabra, POS_tag), pero que pueden incluir errores como "FRASL".
 - `senses`: Lista de sentidos etiquetados (ej. HARD1, HARD2, HARD3).
 - `position`: Índice de la palabra objetivo en el contexto.
- `isinstance(elem, tuple)`: Verifica que cada elemento del contexto sea una tupla.
- `len(elem) == 2`: Asegura que la tupla tenga exactamente dos componentes (palabra y etiqueta POS).
- `all(...)`: Devuelve True solo si todos los elementos del contexto cumplen ambas condiciones, garantizando que no haya elementos mal formateados (ej. cadenas como "FRASL").
- Salida: Booleano (True si la instancia es válida, False si no).

Contexto:

Esta función es una implementación más modular y reusable del filtrado visto en códigos anteriores (ej. `hard_filtrado = [i for i in hard_instances if isinstance(i.context, list) and all(isinstance(x, tuple)`

and len(x) == 2 for x in i.context))). Aquí, la verificación se encapsula en una función dedicada, lo que mejora la legibilidad y permite reutilizarla para otras palabras (ej. "serve").

Línea 5: Filtrado de instancias

Este código crea una lista `hard_clean` que contiene solo las instancias de "hard" con contextos bien formateados, aplicando la función `es_valida`.

Código:

```
hard_clean = [inst for inst in hard_instances if es_valida(inst)]
```

Detalles:

- `hard_instances`: Lista de instancias cargadas con `senseval.instances('hard.pos')`, que contiene 4333 instancias para "hard" (3455 HARD1, 502 HARD2, 376 HARD3, según análisis previos).
- `es_valida(inst)`: Filtra las instancias, excluyendo las 19 identificadas previamente como mal formateadas (con "FRASL" en el contexto).
- `hard_clean`: Lista resultante con aproximadamente 4314 instancias válidas (4333 - 19).
- La comprensión de lista itera sobre `hard_instances` y solo incluye instancias donde `es_valida` devuelve True.

Contexto:

Este paso corresponde a la etapa de limpieza de datos del laboratorio, asegurando que solo se procesen instancias válidas para la extracción de características y el entrenamiento de los clasificadores Naive Bayes. Es equivalente al filtrado visto en el código previo para "hard" y "serve", pero más modular al usar una función dedicada.

Paso 12: Instancias de 'hard' con sentido 'HARD1' mal clasificadas

```
[21]: print("Instancias de 'hard' con sentido 'HARD1' mal clasificadas:\n")

encontradas = 0
for inst, (features, label_gold) in zip(hard_clean[cut:], test_set_c):
    if label_gold == 'HARD1':
        pred = clf_colocacion.classify(features)
        if pred != 'HARD1':
            encontradas += 1
            print("Contexto:", " ".join(word for word, tag in inst.context))
            print(f"Sentido real: {label_gold}, Sentido predicho: {pred}")
            print("-" * 80)

if encontradas == 0:
    print("Ninguna instancia 'HARD1' fue mal clasificada en este conjunto de prueba.")
```

Este código analiza instancias de "hard" con sentido HARD1 mal clasificadas por el clasificador Naive Bayes basado en colocaciones, pero la salida indica que no hay errores, lo cual contradice la matriz de confusión previa (13 errores). Esto sugiere una posible desalineación de datos o una división

diferente. El código es útil para inspeccionar errores, pero requiere verificaciones de consistencia. La alta precisión de HARD1 es consistente con su frecuencia, pero el análisis de errores es clave para mejorar el modelo, como se propuso en el laboratorio. A continuación, se desglosa cada parte del código:

Línea 1: Mensaje inicial

Este código imprime un encabezado para indicar que se mostrarán las instancias de "hard" con el sentido HARD1 que fueron clasificadas incorrectamente.

Código:

```
[21]: print("Instancias de 'hard' con sentido 'HARD1' mal clasificadas:\n")
```

Contexto:

Este mensaje prepara al usuario para un análisis detallado de errores, que es útil para entender las limitaciones del clasificador y explorar posibles mejoras, como se sugirió en el laboratorio (ej. incorporar etiquetas POS o embeddings).

Líneas 3-9: Bucle para identificar instancias mal clasificadas

Este código itera sobre las instancias del conjunto de prueba, identifica aquellas con el sentido HARD1 que fueron mal clasificadas, e imprime su contexto y predicción para análisis.

Código:

```
encontradas = 0
for inst, (features, label_gold) in zip(hard_clean[cut:], test_set_c):
    if label_gold == 'HARD1':
        pred = clf_colocacion.classify(features)
        if pred != 'HARD1':
            encontradas += 1
            print("Contexto:", " ".join(word for word, tag in inst.context))
            print(f"Sentido real: {label_gold}, Sentido predicho: {pred}")
            print("-" * 80)
```

Detalles:

- encontradas = 0: Inicializa un contador para rastrear el número de instancias mal clasificadas.
- for inst, (features, label_gold) in zip(hard_clean[cut:], test_set_c): Itera simultáneamente sobre:
- hard_clean[cut:]: Subconjunto de prueba de las instancias filtradas de "hard" (hard_clean, ~4314 instancias, con ~863 en el conjunto de prueba tras la división 80/20, donde cut ≈ 3451).
- test_set_c: Conjunto de prueba para el modelo de colocaciones, una lista de tuplas (características, etiqueta) generada previamente con extraer_colocacion.

- zip asegura que cada instancia (inst) se alinee con su correspondiente par (features, label_gold) en test_set_c.
- if label_gold == 'HARD1': Filtra las instancias cuyo sentido real es HARD1 (el más frecuente, con ~3455 instancias en el corpus total y ~686 en el conjunto de prueba, según la matriz de confusión previa).
- pred = clf_colocacion.classify(features): Usa el clasificador Naive Bayes basado en colocaciones (clf_colocacion) para predecir el sentido de la instancia.
- if pred != 'HARD1': Identifica instancias mal clasificadas (predicción diferente a HARD1).
- encontradas += 1: Incrementa el contador de errores.
- print("Contexto:", " ".join(word for word, tag in inst.context)): Muestra el contexto de la instancia como una cadena de palabras, ignorando las etiquetas POS.
- print(f"Sentido real: {label_gold}, Sentido predicho: {pred}"): Muestra el sentido real (HARD1) y el predicho (ej. HARD2 o HARD3).
- print("-" * 80): Imprime una línea separadora para claridad.

Contexto:

Este bucle implementa un análisis de errores específico para HARD1, complementando la matriz de confusión previa (que mostró 4 instancias de HARD1 predichas como HARD2 y 9 como HARD3). Es útil para inspeccionar manualmente los contextos que confunden al clasificador.

Líneas 11-12: Mensaje si no hay errores

Este código imprime un mensaje si no se encontraron instancias de HARD1 mal clasificadas.

Código:

```
if encontradas == 0:
    print("Ninguna instancia 'HARD1' fue mal clasificada en este conjunto de prueba.")
```

Salida observada:

```
Instancias de 'hard' con sentido 'HARD1' mal clasificadas:
Ninguna instancia 'HARD1' fue mal clasificada en este conjunto de prueba.
```

Interpretación:

- Contrariamente a la matriz de confusión previa, que mostró 13 instancias de HARD1 mal clasificadas (4 como HARD2, 9 como HARD3), esta salida indica que ninguna instancia de HARD1 fue mal clasificada.
- Esta discrepancia sugiere un posible error o diferencia en los datos:
- Posibilidad 1: El conjunto de prueba (test_set_c) usado aquí no es el mismo que en la evaluación previa, quizás debido a un barajado diferente (el código anterior para "hard" usó random.seed(42), pero no está claro si se aplicó aquí).
- Posibilidad 2: Un error en la generación de test_set_c o en la alineación de hard_clean[cut:] y test_set_c (ej. índices desalineados en zip).

- Posibilidad 3: La matriz de confusión previa refleja un experimento diferente (ej. diferente división de datos).
- Dado que HARD1 tiene alta precisión (~98%, 673/686 correctas), es plausible que en una división específica no haya errores, pero la probabilidad es baja ($13/686 \approx 1.9\%$ de errores esperados).

Contexto:

La ausencia de errores reportada es inesperada, lo que sugiere la necesidad de verificar la consistencia de los datos o el proceso de evaluación.

CONCLUSIONES DE LABORATORIO

Este laboratorio tuvo como objetivo la desambiguación semántica supervisada de las palabras “**hard**” (con sentidos HARD1, HARD2, HARD3) y “**serve**” (con sentidos SERVE10, SERVE12, SERVE2, SERVE6), utilizando el corpus etiquetado **Senseval 2**. Para ello, se compararon dos enfoques de clasificación Naive Bayes:

- Basado en **palabras vecinas** (modelo de bolsa de palabras).
- Basado en **colocaciones** (bigramas contextuales previos y siguientes).

Ambos clasificadores fueron evaluados en términos de exactitud, errores comunes, matriz de confusión y comparación con líneas base aleatorias y mayoritarias. Y es por esto por lo que lo primero que determinamos fueron las limitaciones de los clasificadores que se basan en:

1. Desbalance de clases

El corpus presenta una distribución altamente desigual entre los sentidos:

- En “hard”, el sentido **HARD1** representa más del 80% de las instancias, lo que llevó a una alta tasa de clasificación errónea de sentidos minoritarios: 43 casos de HARD2 y 39 de HARD3 fueron mal etiquetados como HARD1.
- En “serve”, los sentidos **SERVE10** y **SERVE12** dominan, mientras que **SERVE6** obtuvo solo un 27% de precisión.

2. Características poco expresivas

- El enfoque de **palabras vecinas** (exactitud: 0.7891 para “hard”, 0.6713 para “serve”) utiliza vocabulario general y descontextualizado.
- El modelo de **colocaciones** (0.8818 y 0.7282 respectivamente) mejora el rendimiento, pero solo considera dos bigramas por instancia, lo que resulta insuficiente para contextos complejos.

3. Ambigüedad contextual

Muchos errores se explican por contextos vagos o genéricos (ej. “*that’s*”, “*do*”), que no aportan suficiente información desambiguadora.

4. Limitación del enfoque supervisado

Los clasificadores solo identifican sentidos previamente etiquetados en Senseval 2, sin capacidad de generalizar a nuevas acepciones.

5. Inconsistencias de evaluación

En una prueba, se reportó incorrectamente que ninguna instancia de HARD1 fue mal clasificada, mientras que la matriz de confusión evidenció 13 errores. Esto refleja problemas de reproducibilidad por falta de semilla aleatoria o desincronización entre los datos de entrenamiento y prueba.

6. Limitaciones de Naive Bayes

La independencia entre características que asume este clasificador impide capturar relaciones semánticas o sintácticas complejas.

Ahora y por otra parte a continuación denotamos algunas alternativas de mejora que hemos podido plantearnos:

1. **Modelos contextuales modernos:** Utilizar embeddings como **BERT** o **RoBERTa**, que comprenden el contexto completo de una oración.
2. **Características sintácticas:** Incorporar etiquetas POS y relaciones de dependencia sintáctica con herramientas como SpaCy.
3. **Aprendizaje no supervisado:** Emplear modelos como **Sentence-BERT** y clustering para identificar sentidos no anotados.
4. **Balanceo de clases:** Aplicar técnicas como **SMOTE** para aumentar instancias de sentidos minoritarios y reducir el sesgo.
5. **Ampliación del corpus:** Incorporar ejemplos de **WordNet** o **SemCor** para representar mejor los sentidos menos frecuentes como SERVE6.
6. **Modelos híbridos:** Combinar bigramas, POS tags y embeddings con clasificadores como **SVM** para mayor rendimiento.
7. **Validación cruzada:** Implementar validación cruzada de 5 pliegues para garantizar resultados consistentes y robustos.

Ahora y para lograr una mejor comprensión a continuación damos a conocer una tabla donde efectuamos la comparación entre los clasificadores:

Palabra	Clasificador	Exactitud	Baseline Aleatoria	Baseline Mayoritaria	Ganancia sobre Aleatoria	Ganancia sobre Mayoritaria
hard	Palabras vecinas	0.7891	0.3333	0.7995	+0.4558	-0.0104
hard	Colocaciones	0.8818	0.3333	0.7995	+0.5485	+0.0823
serve	Palabras vecinas	0.6713	0.2500	0.4216	+0.4213	+0.2497
serve	Colocaciones	0.7282	0.2500	0.4216	+0.4782	+0.3066

Es por esto y como una reflexión sobre esta comparación debemos indicar que comparar directamente la exactitud de los clasificadores para "hard" y "serve" no es adecuado, debido a diferencias clave, la primera es que **"hard"** tiene 3 sentidos, mientras que **"serve"** tiene 4, lo que

aumenta la complejidad. Adicionalmente, HARD1 (80%) domina significativamente más que SERVE10 (42%), lo que sesga los resultados. Y por último los sentidos de “serve” presentan mayor solapamiento semántico, dificultando la clasificación.

Ahora y para una comparación justa se recomienda usar **ganancia sobre baseline**, no solo exactitud bruta, además calcular **F1-score por clase** para evaluar equilibrio y en adición normalizar los resultados respecto al número de clases para balancear los datos de entrenamiento y prueba.

Como conclusión general debemos decir que este laboratorio demostró que el enfoque basado en **colocaciones** supera en rendimiento al de **palabras vecinas**, con mejoras notables en exactitud para ambas palabras analizadas. Sin embargo, el **desbalance de datos**, la **simplicidad de las características** y las **limitaciones del clasificador Naive Bayes** afectan especialmente la clasificación de sentidos minoritarios como HARD3 y SERVE6.

Asimismo, la inconsistencia detectada en la clasificación de HARD1 resalta la importancia de garantizar la **reproducibilidad** mediante semillas aleatorias y validaciones cruzadas. Las mejoras propuestas, incluyendo el uso de modelos basados en contexto (como BERT), el balanceo de datos, y la integración de características sintácticas, ofrecen un camino sólido para avanzar en tareas de desambiguación semántica más robustas y generalizables.

BIBLIOGRAFÍA

A continuación, la bibliografía implementada en la búsqueda de información:

- Tema 5. Modelado del lenguaje. Tema 6. Modelado del Lenguaje Neuronal. Tema 7. Análisis semántico. Tema 8. Semántica léxica. Tema 9. Aplicaciones del procesamiento del lenguaje natural. Tema 10. Agentes conversacionales.
- Clases virtuales con el profesor Ing. Diego Osorio Reina.
- Material de apoyo del aula ubicado en Documentación.
- Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit. O'Reilly Media.
- Jurafsky, D., & Martin, J. H. (2023). Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition (3rd ed. draft). <https://web.stanford.edu/~jurafsky/slp3/>
- Navigli, R. (2009). Word Sense Disambiguation: A Survey. ACM Computing Surveys, 41(2), 1-69. <https://doi.org/10.1145/1459352.1459355>
- Pedersen, T. (2002). Evaluating the Effectiveness of Ensembles of Decision Trees in Disambiguating Senseval Lexical Samples. Proceedings of the First International Workshop on Evaluating Word Sense Disambiguation Systems (Senseval-2), 81-84.
- Manning, C. D., & Schütze, H. (1999). Foundations of Statistical Natural Language Processing. MIT Press.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 4171-4186. <https://doi.org/10.18653/v1/N19-1423>
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321-357. <https://doi.org/10.1613/jair.953>
- NLTK Project. (2023). NLTK Documentation. <https://www.nltk.org/>
- Agirre, E., & Edmonds, P. (Eds.). (2007). Word Sense Disambiguation: Algorithms and Applications. Springer.
- Hovy, E., Marcus, M., Palmer, M., Ramshaw, L., & Weischedel, R. (2006). OntoNotes: The 90% Solution. Proceedings of the Human Language Technology Conference of the NAACL, 57-60. <https://doi.org/10.3115/1220835.1220842>
- SpaCy. (2023). Industrial-Strength Natural Language Processing in Python. <https://spacy.io/>

- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... & Rush, A. M. (2020). Transformers: State-of-the-Art Natural Language Processing. Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 38-45. <https://doi.org/10.18653/v1/2020.emnlp-demos.6>
- McCallum, A., & Nigam, K. (1998). A Comparison of Event Models for Naive Bayes Text Classification. AAAI-98 Workshop on Learning for Text Categorization, 41-48.
- Mihalcea, R., & Moldovan, D. (2001). Automatic Generation of a Coarse Grained WordNet. Proceedings of the NAACL Workshop on WordNet and Other Lexical Resources, 35-41.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.